TABLE OF CONTENTS

목차

PORTFOLIO · 김단비 · FULLSTACK

01

COVER

커버 — 풀스택 Portfolio 메인

02

01 · EXPERIENCE

실무 경력 — 2년의 운영 오너십

03

02 · PROJECT

프로젝트 #1 — 다2조부르릉

04

03 · PROJECT

프로젝트 #2 — ants-camp

05

04 · CAPABILITY

AI 워크플로우 — 하네스 엔지니어링

PORTFOLIO · 김단비 · FULLSTACK

서버에서 시작해 화면까지 잇는 기능의 시작부터 끝까지 책임지는 풀스택.

회원제 SaaS 운영팀에서 **뷰어 UX·동시성 제어·RDBMS 마이그레이션**을 화면과 서버 양쪽에서 직접 풀었고,
최근 프로젝트에서는 **RAG 백엔드부터 React 운영 콘솔까지** 한 기능을 끝까지 구현했습니다.
사용자가 체감하는 문제를 시스템 구조로 해결하고, "왜 이 기술을 쓰는가"를 끝까지 묻습니다.

● github.com/danbeekimm ● velog.io/@danbeekimm ▶ ants-camp 라이브 데모 ▶ 다2조부르릉 라이브 데모

WORKS

프로젝트 · 경력

세 작업을 시간순으로. **화면과 서버 양쪽에서 사용자 문제를 풀어 온 궤적** — 뷰어 UX·동시성을 실무에서 풀고, 부트캠프에서 RAG 백엔드부터 React 운영 콘솔까지 한 기능을 끝까지 구현했습니다.

💼 EXPERIENCE · 실무 경력 2022.11 ~ 2025.03

뷰어 UX와 서버 동시성을 양쪽에서 풀어낸 2년

풀스택 개발자 — 회원제 특허 SaaS의 고도화·운영

대량 문서 뷰어의 **도면-텍스트 실시간 동기화**를 부분 렌더링 → 비동기 갱신으로 재설계해 깜빡임·포커스 강탈을 제거하고,
서버에서는 분할 비동기 다운로드의 **이용량 우회 race condition**을 "임계구역 위치 재배치"로, **PostgreSQL → MSSQL 전면 마이그레이션**의 슬로우 쿼리를 옵티마이저 단위로 풀었습니다.
화면에서 느낀 한계가 서버 설계로, 다시 부트캠프 학습으로 이어졌습니다.

만족도 2위	원천 차단	동일 수준
뷰어 동기화 UX 개선 · 연간 조사	다중 창 동시 요청 한도 우회	벤더 변경 후 응답 성능 유지

`JSP · AngularJS · jQuery` `비동기 렌더링 · 스크롤 동기화` `Spring` `MyBatis` `MSSQL · T-SQL UDF` `비동기 동시성`

담당: **6개 검색 사이트 유지보수·개선 — 대량 문서 뷰어 · 다운로드 · 비교보기**

상세 보기→

🚚 PROJECT #1 · 다2조부르릉 6인

정합성 보장과 장애 격리를 설계한 B2B 물류 MSA

12개 마이크로서비스 · DDD 4계층 헥사고날 · "검증=동기, 통지=비동기" 통신 분리

직전 모놀리식의 한계(한 도메인 결함이 전체 멈춤)를 계기로 **MSA + DDD**로 확장.
분산 트랜잭션 없이 **Outbox 패턴**으로 이벤트 정합성을 보장하고, **OR-Tools VRPTW**로 B2B 시간창 SLA를 제약 조건으로 모델링했습니다.
비관적 락은 스터디에서 학습한 트레이드오프를 실제 적용한 사례.

at-least-once	5분 → 3초	DB-level
Outbox · 결과적 일관성	VRPTW 자동 배정 (100건 · 데모)	비관적 락으로 race 차단

`Spring Cloud 2025` `Kafka · Outbox` `OpenFeign` `Resilience4j CB` `OR-Tools VRPTW` `QueryDSL` `PESSIMISTIC_WRITE`

담당: **company-service 전담 + DeliveryManager 도메인**

▶ 라이브 데모 da2joburureung 상세 보기→

🤖 PROJECT #2 · ants-camp 5인

RAG 백엔드부터 React 운영 콘솔까지 끝까지 구현

모의투자 MSA · LLM 정량 평가 · AIOps · 프론트엔드 전체 구축

LLM-as-a-Judge + Pairwise로 모델 교체를 데이터로 결정하고, Prometheus·Loki·Claude를 묶어 AIOps를 구성했습니다.
나아가 **그 평가·운영 결과를 다루는 관리자 콘솔을 React·TS로 직접 구현** — 백엔드 API를 화면이 필요로 하는 형태로 설계하고, 사용자 화면(시세·매매·챗봇)까지 프론트엔드 전체를 맡았습니다.

2.38 → 4.04	64.3% → 8.3%	프론트엔드	74% ↓
faithfulness 향상 · 모델 교체	환각률 감소	사용자 + 관리자 화면 구축 주도	캐시 히트 응답 단축 · LLM 호출 스킵

`Spring AI 1.0` `pgvector` `LLM-as-a-Judge` `React · TypeScript` `Tailwind · Zustand` `Prometheus · Loki` `Kafka · Redis`

담당: **assistant · notification · monitoring + 프론트엔드 (React·TS) 구축 주도**

▶ 라이브 데모 frontend backend 상세 보기→

🧠 CAPABILITY · AI 워크플로우 상시

AI에게 일임하지 않고 통제한다 — 하네스 엔지니어링

harness engineering · 규칙 주입 · 가드레일 · 컨텍스트 설계

개발 사이클 단계마다 도구를 나눠 쓰되 **설계·결정과 마지막 판단은 항상 사람이** 합니다.
코딩 규칙·아키텍처 패턴을 **SSOT 문서(CLAUDE.md·컨벤션)로 못박아** AI가 규칙을 벗어나지 못하게 하고, 작업이 끝나면 **/revref·/apitest가 자동으로 검토·검증**해 AI의 실수를 걸러냅니다. AI에게 개발을 맡기는 게 아니라, **AI를 관리·감독하는 워크플로우를 직접 엔지니어링**합니다.

단계별 분리	자동 검토	규칙 흡수
분석 · 구현 · 리뷰 도구 분리	/revref · CodeRabbit 컨벤션 점검	반복 지적 → 규칙 문서 → 재발 차단

`Claude Code` `서브에이전트` `CLAUDE.md · 컨벤션` `/revref` `/apitest` `CodeRabbit`

원칙: **AI에게 개발을 일임하지 않는다 — 통제하고 감독한다**

상세 보기→

SKILLS

기술 스택

	💼 실무 경험 (2년)	🎓 학습 · 프로젝트
LANGUAGE	Java 8 · JavaScript	Java 17 · TypeScript
BACKEND FW	Spring 4.x (Legacy)	**Spring Boot 3.5** · **Spring Cloud** (Gateway · Eureka · Config · OpenFeign) · **Spring AI 1.0** · Spring Security
FRONTEND	JSP · AngularJS · jQuery — 비동기 렌더링 · 스크롤·이미지 동기화	**React** · **TypeScript** · **Tailwind CSS** · Zustand · Recharts (ants-camp 화면 전체 구축)
DATA ACCESS	MyBatis · Stored Procedure (T-SQL)	Spring Data JPA · QueryDSL
DATABASE	MSSQL · MySQL · PostgreSQL	PostgreSQL (+**PostGIS · pgvector**) · Redis
ARCHITECTURE	모놀리식 (대규모 운영)	**MSA · DDD · Hexagonal · Event-Driven**
MESSAGING / RESILIENCE	—	Apache Kafka · **Outbox Pattern** · Resilience4j (Circuit Breaker · Retry)
OBSERVABILITY	운영 로그 분석 · CS 패턴 추적	Prometheus · Grafana · Loki · Promtail · Alertmanager · Zipkin
AI / LLM	—	Spring AI · RAG · pgvector · **LLM-as-a-Judge** (Eval · Pairwise) · **LLM 기반 AIOps** · Human-in-the-Loop
OPTIMIZATION / PARSING	Apache POI (대용량 문서 파싱 · DRM 시그널 식별)	Google OR-Tools (VRPTW)
TEST	JUnit · Mockito (팀 도입 주도)	JUnit · Mockito
REFACTORING / CODE QUALITY	**SonarQube** (월간 코드리뷰·리팩토링 운영)	—
DEVOPS	—	Docker · Docker Compose · GitHub Actions
CLOUD / DEPLOY	—	**GCP** (Compute Engine · Cloud SQL · Container Registry) · **AWS**

김단비 · 풀스택 개발자 — Portfolio v2026.05

← 포트폴리오 메인으로

01 · OVERVIEW · 실무 경력

이슈를 끝까지 추적하는 운영 오너십

회원제 특허 SaaS의 **6개 검색 사이트 유지보수·개선 담당**으로, **대량 문서 뷰어·엑셀 다운로드·비교보기**를 메인으로 담당했습니다. 운영하면서 마주친 **코드·기획·데이터의 불일치**를 그냥 넘어가지 않고 원인까지 추적해 해결하는 방식으로 일했습니다. 담당 기능에 머무르지 않고 **서비스의 상태 자체를 책임진다는 태도**로 운영을 대했습니다.

"이슈를 발견하면 넘어가지 않고, 보고하고, 해결까지 책임졌다."

대량 데이터를 다루는 회원제 SaaS — 6개 검색 사이트의 유지보수·개선. 대량 문서 뷰어·엑셀 다운로드·비교보기를 메인으로, 그 외 사용자 편의성·로그 분석·외부 API 연동·DB 마이그레이션까지 담당 영역을 좁게 한정하지 않고 서비스 전체의 상태를 능동적으로 관리했습니다.

- 풀스택 개발자
- 2022.11 ~ 2025.03

🔍

이슈를 그냥 넘기지 않는다

코드 동작이 어색하거나 데이터가 이상하면 **원인을 직접 추적**. 표면 증상에서 멈추지 않고 근본 원인까지 들고 가서 해결.

🤝

기획 단계부터 함께 협업한다

기획 자문 · 사용자 관점 의견 제시 · 코드 불일치 검토까지 함께 맡음. 기획팀과 활발히 소통하며 더 나은 방향을 같이 찾음.

🏠

운영을 내 것으로 본다

CS 문의·로그·이상 패턴을 능동적으로 모니터링하며 **서비스 전체의 상태**를 지켜봄.

★ MAIN ROLE

6개 검색 사이트 유지보수·개선

담당 기능: **대량 문서 뷰어(이지뷰어) · 엑셀 다운로드 · 비교보기**

★ 이지뷰어 도면-텍스트 동기화 작업이 연간 사용자 만족도 조사에서 2위로 선정됨

+ ADDITIONAL

유틸리티 · 편의성 개선

형광펜 · DRM 필터링 · 메인-서브 뷰어 연동 · 다운로드 한도 모니터링

+ ADDITIONAL

분석 · 연동 · 마이그레이션

로그 기반 사용자 행동 분석 · 외부 번역 API 연동 · PostgreSQL → MSSQL 마이그레이션

🛠 코드 수정 루틴 — 유지보수 환경에서 익힌 작업 원칙

STAGE 01

변경 전 — 코드만 보지 않는다

- ✓ **SVN history**로 과거 결정 맥락 추적 — 이 코드가 왜 이렇게 짜였는지부터 확인
- ✓ **기획서·과거 CS 이슈** 확인 — *같은 문제 두 번 풀지 않기*
- ✓ DB 직접 조회로 **코드가 가정한 데이터**와 **실제 데이터**의 간극 확인

STAGE 02

변경 중 — 추측 대신 재현

- ✓ **버그 재현 시나리오부터 확정** — "이 조건에서 100% 재현"을 만든 뒤 작업 시작
- ✓ 디버거 step-by-step으로 흐름 검증 — *코드 읽고 추론하지 않음*

STAGE 03

변경 후 — 내 일이 끝나는 시점을 늦춘다

- ✓ **운영 로그 직접 확인** — 변경 직후 며칠간 이상 패턴 추적
- ✓ **관련 CS 인입 의식적으로 추적** — *배포 후 며칠간을 작업의 마무리 구간으로 둠*

일관된 원칙: *"맥락을 추적한다 + 추측 대신 재현으로 확인한다 + 내 일이 끝나는 시점을 며칠 뒤로 미룬다." 이 셋이 재직 기간 동안 큰 운영 사고 없이 작업을 유지한 실질적 근거.*

02 · HOW I WORK

이슈를 발견·추적·해결한 방식

담당 기능의 경계를 좁게 두지 않았습니다. **운영 중 이상한 점 → 보고 → 원인 추적 → 해결**을 능동적으로 반복했고, 특히 **기획과 코드의 불일치**를 발견하면 기획팀과 직접 의논해 정정한 경험이 많습니다.

🤝 CASE #1

기획 단계부터 함께한 협업 — 자문 · 사용자 관점 · 코드 불일치 검토

기획과 사용자 사이를 잇는 역할로 일하기

기획 자문	기획 초기 단계에서 **시스템적으로 가능한 범위 · 비용 · 우회 방법**에 대한 자문을 요청받음. 받은 기획을 그대로 구현하기보다, 이 방향이 운영에 어떤 부담을 줄지를 함께 검토.
사용자 관점	**실제 사용자가 되어 서비스를 써보며** 불편한 지점을 직접 체감. 기획에 적힌 흐름과 실제 사용 경험 사이의 간극을 발견하면 의견을 정리해서 공유. "사용자 입장에서 편리한가"를 기준으로 기획팀과 활발히 소통.
코드 불일치 검토	새 기획안이 나오면 받자마자 **"이 구조가 지금 코드와 맞나?"**를 먼저 확인. 기획팀이 알고 있는 시스템 모습과 실제 운영 중인 코드의 동작이 다른 경우를 정리해서 공유 — 어느 쪽을 정정할지 의논해서 결정.
결과	"막상 만들고 보니 의도와 다르다"는 재작업 비용을 사전에 줄이고, 기획팀도 시스템의 실제 모습을 더 정확히 알게 됨. 기획·사용자·코드 사이를 잇는 역할도 함께 맡았습니다.

🔍 CASE #2

반복 CS 문의의 원인 분석 → 팀 논의 → 사전 차단 구현

문제의 단서를 제공하고, 결정된 방향을 책임지고 완수한 사례

상황	"문서 업로드 시 오류가 발생한다"는 사용자 문의가 반복 접수. 1차 대응은 CS팀이 "E-DRM 해제 후 재업로드"를 수동 안내하는 방식이었음.
팀 결정 · 할당	"수동 안내 대신 시스템 레벨에서 사전 차단하자"는 방향이 팀에서 결정됨. 해당 작업이 본인에게 할당됨.
원인 분석	반복되는 Exception 로그를 분석해, 사내 E-DRM 암호화 문서를 업로드할 때 Java 기반 문서 파서들이 던지는 **특정 Exception 패턴**이 존재함을 식별. E-DRM 암호화로 문서의 바이너리 구조 자체가 변형되어 파서가 정상 문서로 인식하지 못하기 때문이라, 무작위 에러가 아니라 일정한 원인이 있는 케이스임을 확인.
구현 · 결과	업로드 단계에서 외부 분석 API에 넘기기 전에 **사전 파싱으로 시그널을 감지**해 "암호화 해제 후 재업로드" 안내 메시지를 노출. 파일 형식별로 도구를 분리해 **MS Office 계열은 Apache POI, HWP는 hwplib**으로 동일 패턴 적용. 일반 파싱 오류와 E-DRM 케이스를 명확히 분기 → 불명확한 Exception 화면이 명확한 안내로 바뀌고, **동일 원인의 CS 문의 재발 없음.**

03 · CONTRIBUTIONS CATALOG

운영 기여 한눈에 보기

메인 업무(**이지뷰어·엑셀 다운로드·비교보기**)부터 마이그레이션·동시성·UX·다국가 출시·팀 문화까지, 운영 환경의 다양한 문제를 직접 해결한 작업들의 카탈로그. **★ DEEP DIVE** 배지가 있는 항목은 본문에서 자세히 다룸.

이지뷰어 ↔ 도면 팝업 실시간 동기화 ★ DEEP DIVE
사용자 만족도 ★ 2위

이지뷰어 문헌 이동 시 별도 팝업 도면을 자동 동기화. **URL 재호출 → 부분 렌더링(동기) → 비동기 + 콜백 재구성**의 3단계로 깜빡임·포커스 강탈·동기 멈춤을 차례로 제거. **연간 사용자 만족도 조사 2위.**

→ 04 · DEEP DIVE — VIEWER UX

PG → MSSQL 슬로우 쿼리 개선 ★ DEEP DIVE
MyBatis · T-SQL

MSSQL 2016에 `string_agg(DISTINCT)` 가 없어 STUFF + FOR XML PATH 우회 → 성능 저하. 서브쿼리·JOIN·WHERE 구조 조정 후 **임시 테이블 단계 분할**로 해결, 기존 환경 대비 동일 응답 성능 유지.

→ 05 · DEEP DIVE — DB MIGRATION

유료 다운로드 한도 우회 차단 동시성 ★ DEEP DIVE

1만 건 분할 비동기 처리에서 다중 창 동시 요청으로 한도 우회 가능했던 race condition을 **사전 차감**으로 해결. 락 기반 대안과 트레이드오프 비교 후 환경에 맞는 도구 선택.

→ 06 · DEEP DIVE — CONCURRENCY

대규모 특허 번역 — fan-out 설계 의사결정 ★ DEEP DIVE
UX 성능

200건 번역 노출 시 **서버 일괄 vs 클라이언트 fan-out**의 트레이드오프(인프라 부하↑ vs 체감 응답성↑)를 정리. 인프라 수용량·페이지 UX 중요도를 근거로 fan-out 채택. 로딩 저하 우려에도 **눈에 띄는 이슈 없이 출시.**

→ 07 · DEEP DIVE — STREAMING UX

E-DRM 암호화 문서 사전 차단 ★ DEEP DIVE
Apache POI · hwplib

반복 CS 문의의 원인 추적 → *E-DRM 암호화로 바이너리 구조가 변형되어 Java 파서가 일관된 Exception을 던지는 패턴*을 식별. 외부 API 호출 전 **Apache POI(MS Office) · hwplib(HWP)로 사전 파싱 검증**해 명확한 안내로 분기. **동일 사유 CS 재발 없음.**

→ 02 · HOW I WORK · CASE #2

EU 지역 서비스 신규 출시 다국가

검증된 타 국가 서비스의 구조·패턴을 분석해, **EU 특화 요구(언어·지역 정책)를 반영하면서도 일관성 있는 구조**로 안정적 출시. 기존 시스템의 추상화 레벨을 유지하며 확장한 사례.

JUnit/Mockito 테스트 도입 주도 팀 문화

인증이 필요한 컨트롤러 테스트 방법론을 정립해 팀 대상 발표 → 팀 표준 가이드라인 마련. 테스트 방법뿐 아니라 그 근거까지 함께 공유.

비교 뷰어 스크롤 동기화 UX 알고리즘

기존 **절대 이동거리 동기화**는 길이가 다른 두 문서에서 어긋남 → **이동거리 비율(0~1) 동기화**로 변경해 양쪽 문서가 항상 같은 상대 위치를 보여줌. "UX 문제를 코드 구조로 푼" 사례.

형광펜 대규모 고도화 UX 데이터 모델

드래깅·리사이징 레이어 도입, 색상 16 → 40+개 확장, 저장 포맷 **XML → JSON** 전환. 데이터 모델 자체를 재설계해 향후 기능 확장을 쉽게 만든 작업.

로그 기반 사용자 행동 패턴 조사 운영 지원

CS·영업팀 요청을 받아 **로그인 시간이 불규칙한 사용자 등 의심 패턴 대상자를 로그 다각 분석으로 조사**해 운영팀에 보고. 이상 로그인 등 의심 패턴을 읽어내는 관점으로 접근.

04 · DEEP DIVE — VIEWER UX

★ DEEP DIVE — VIEWER UX

이지뷰어 ↔ 도면 팝업 실시간 동기화 — 연간 사용자 만족도 ★ 2위

메인 담당 기능의 핵심 사용성 작업. 단계별로 문제 원인을 짚어 개선한 결과, 그 해 전체 개선 기능 중 사용자 만족도 2위로 선정

P · 메인 창에는 다수의 특허 문헌을 빠르게 탐색할 수 있는 **"이지뷰어"**가 있고, **"도면 보기"**를 클릭하면 별도 컨트롤러 URL을 호출해 전체 HTML을 받아오는 **팝업 창**이 열리는 구조. *A문헌 도면을 본 상태에서 이지뷰어로 다른 문헌을 탐색해도 팝업은 A문헌 도면 상태로 고정*되어, 사용자가 매번 "도면 보기"를 다시 클릭해야 하는 불편이 있었음.

A · 이지뷰어 문헌 이동 시 팝업 도면이 자동 동기화되도록 개선. **각 단계에서 드러난 문제 원인을 정확히 짚어 다음 단계로 이어감.**

> **1차 — 팝업 자동 갱신 (URL 재호출)**
>
> B문헌으로 이동 시 도면 조회 컨트롤러 URL을 다시 호출해 **팝업 창 전체를 새로 불러오는 방식.**
>
> *남은 문제 3가지: ① 페이지 전체 재로딩으로 느림 ② 화면이 순간 하얗게 깜빡임 ③ 새 URL 로드 시 **브라우저 보안 정책에 의해 포커스가 팝업 창으로 강제 이동** → 이지뷰어에서 빠르게 문헌을 넘기던 탐색 흐름이 매번 끊김.*

> **2차 — 부분 렌더링 (동기 AJAX)**
>
> 깜빡임·포커스 강탈의 근본 원인이 **"팝업 URL을 바꿔 페이지 전체를 재로딩"하는 데 있다고 판단**. 팝업 HTML을 **고정 영역(헤더·툴바) + 가변 영역(도면)**으로 분할하고, 팝업 URL은 그대로 둔 채 **가변 영역의 HTML 조각만 동기 AJAX로 받아 innerHTML로 교체.**
>
> *해결: 깜빡임·포커스 강탈 사라지고 전송량 감소로 속도 개선. 남은 문제: 동기 AJAX 특성상 응답 도착 전까지 메인 스레드가 멈춰 "순간 멈춤"이 인지됨.*

> **3차 — 비동기 호출 + 콜백 기반 실행 순서 재구성**
>
> AJAX를 비동기로 변경. 단순히 동기→비동기 전환만으로는 부족하고, **응답 시점에 의존하는 후속 로직(도면 초기화 JS 등)이 응답 완료 후 실행되도록 콜백 기반으로 코드 구조를 재정비.**
>
> *해결: 응답 대기 중에도 사용자 입력에 반응 가능해져 멈춤 현상까지 해소. 1·2차에서 드러난 모든 문제(깜빡임·포커스 강탈·속도·멈춤)가 최종적으로 모두 개선됨.*

R · **연간 사용자 만족도 조사에서 그 해 개선 기능 중 2위로 선정.** *기능이 동작하는 데 멈추지 않고 사용자 흐름을 끊지 않는 것까지 가져간 점*이 실질적 평가 근거.

> 🔄 회고
>
> *AJAX 적용 자체*보다, 각 단계의 문제 원인을 정확히 짚어 다음 단계로 이어간 점이 중요했음. **페이지 전체 재로드 → 부분 교체 → 비동기화**로의 흐름은 결과적으로 **"브라우저가 강제하는 동작을 피하기 위해 점점 더 가벼운 단위로 갱신 범위를 좁혀 나간 과정"**이었음.

→ 작업의 의미

브라우저가 강제하는 동작 단위를 좁혀 사용자 흐름을 보존하는 설계. 사용자 경험을 코드 구조 레벨에서 직접 풀어낸 작업.

점진적 응답성 개선(전체 재로딩 → 부분 렌더링 → 비동기 갱신)의 사고 패턴은 이후 **대규모 특허 번역**(서버 일괄 응답 → 클라이언트 fan-out + 점진 교체)으로 이어지고, 부트캠프 프로젝트의 *ants-camp 클라이언트 fan-out 구조*까지 일관되게 적용됨.

05 · DEEP DIVE — DB MIGRATION

🗄 DEEP DIVE — DB MIGRATION

PostgreSQL → MSSQL 마이그레이션 후 슬로우 쿼리 개선

시행착오로 해결한 작업을 이후 학습으로 원리 레벨에서 설명할 수 있게 된 사례

P · 팀 차원에서 PG → MSSQL(구버전, 2016) 전체 이관을 진행. 마이그레이션 후 **특정 쿼리가 눈에 띄게 느려진 케이스**를 담당해 개선. *특히 주된 원인은, 중복 제거 문자열 집계에서 PG의 `string_agg(DISTINCT)` 가 MSSQL 2016에는 없어 **STUFF + FOR XML PATH + DISTINCT 서브쿼리**로 우회해야 했고, 이 우회 자체가 큰 성능 저하를 유발.*

A · 같은 결과를 내는 여러 SQL 형태를 시도하며 단계적으로 접근:

▾ SQL 형태별 비교 시도 ①~④ — 시행착오 과정 클릭해 펼치기

> **① 호출 경로 식별 — 영향 범위 먼저 파악**
>
> 슬로우 쿼리가 발생하는 화면·기능을 먼저 식별해 회귀 검증 범위를 가늠.
>
> *이유: 쿼리만 보고 고치면 같은 쿼리가 호출되는 다른 화면에서 회귀가 날 수 있음.*

> **② 서브쿼리 ↔ JOIN ↔ UNION 변환 시도**
>
> 서브쿼리로 짠 부분을 JOIN으로, JOIN을 UNION으로 바꿔보며 MSSQL에서 어떤 형태가 빠른지 실행 시간 측정으로 비교.
>
> *결과: 일부 쿼리는 개선, 일부는 미미.*

> **③ WHERE절 구조 조정**
>
> 조건 순서·표현 방식을 다양하게 시도.
>
> *결과: 미미한 개선. 근본 해결은 안 됨.*

> **④ 임시 테이블로 단계 분할 — 결정적 해결**
>
> 복잡한 한 방 쿼리를 단계별로 쪼개서 임시 테이블에 넣고 다음 단계에서 사용.
>
> *결과: 큰 개선 확인. 이게 결정적이었음.*

R · 벤더가 바뀌어도 **기존 환경 대비 동일 수준의 응답 성능 유지**하며 슬로우 쿼리 개선 완료.

> 📚 당시 한계와 이후 학습
>
> 당시엔 옵티마이저 개념을 깊이 알지 못한 채 시행착오로 해결. **이후 학습 결과로 이해한 메커니즘:** MSSQL 옵티마이저는 참여 테이블 수와 데이터 분포가 복잡해질수록 통계 추정이 빗나가 *잘못된 조인 순서나 비효율적 조인 방식(예: 해시 조인이 적절한데 중첩 루프를 택함)*을 선택하는 경향이 있음. 거대한 단일 쿼리를 통째로 던지면 옵티마이저가 길을 잃는 상황이었던 것. 임시 테이블 분할은 각 단계마다 데이터 크기·통계가 명확한 상태에서 옵티마이저가 판단하도록 강제하는 효과. **다음 작업 시 접근법:** 실행 계획을 먼저 확인하고 옵티마이저의 추정·실제 차이를 진단한 뒤 접근.

→ 학습 연결 고리

이 경험에서 **SQL 의존성·벤더 종속성의 한계**를 체감 → **ORM(JPA) 학습**으로 직결 → 영속성 컨텍스트·벤더 독립 데이터 접근 계층 설계를 학습 → 부트캠프 프로젝트의 **JPA + QueryDSL 설계**에 적용.

06 · DEEP DIVE — CONCURRENCY

⚡ DEEP DIVE — CONCURRENCY

임계구역을 긴 로직 뒤에서 맨 앞으로 옮긴 비동기 동시성 설계

1만 건 분할 비동기 처리에서 다중 창 동시 요청을 통한 한도 우회 원천 차단 + 비동기 모니터링 누락 제거

P · 1만 건 규모 다운로드를 100건씩 분할 비동기 처리하는 회원제 기능에서, 이용량을 **모든 처리 완료 후** 집계하는 구조 → 사용자가 여러 창에서 동시에 다운로드하면 **검사 시점에 한도가 비어있어 제한 우회**(race condition). 비동기 특성상 실패 다운로드의 모니터링 알림도 누락.

A · 임계구역의 위치 자체를 재설계 — 옵션별로 트레이드오프 비교:

대안 검토 — 동시성 보호 옵션 비교

REJECTED · DB 락

SELECT ... FOR UPDATE

비동기 처리 시간(수 초~수십 초) 동안 락 점유 → **처리량 저하 + 다른 정상 요청까지 블로킹.**

REJECTED · 분산 락

Redis · ZooKeeper

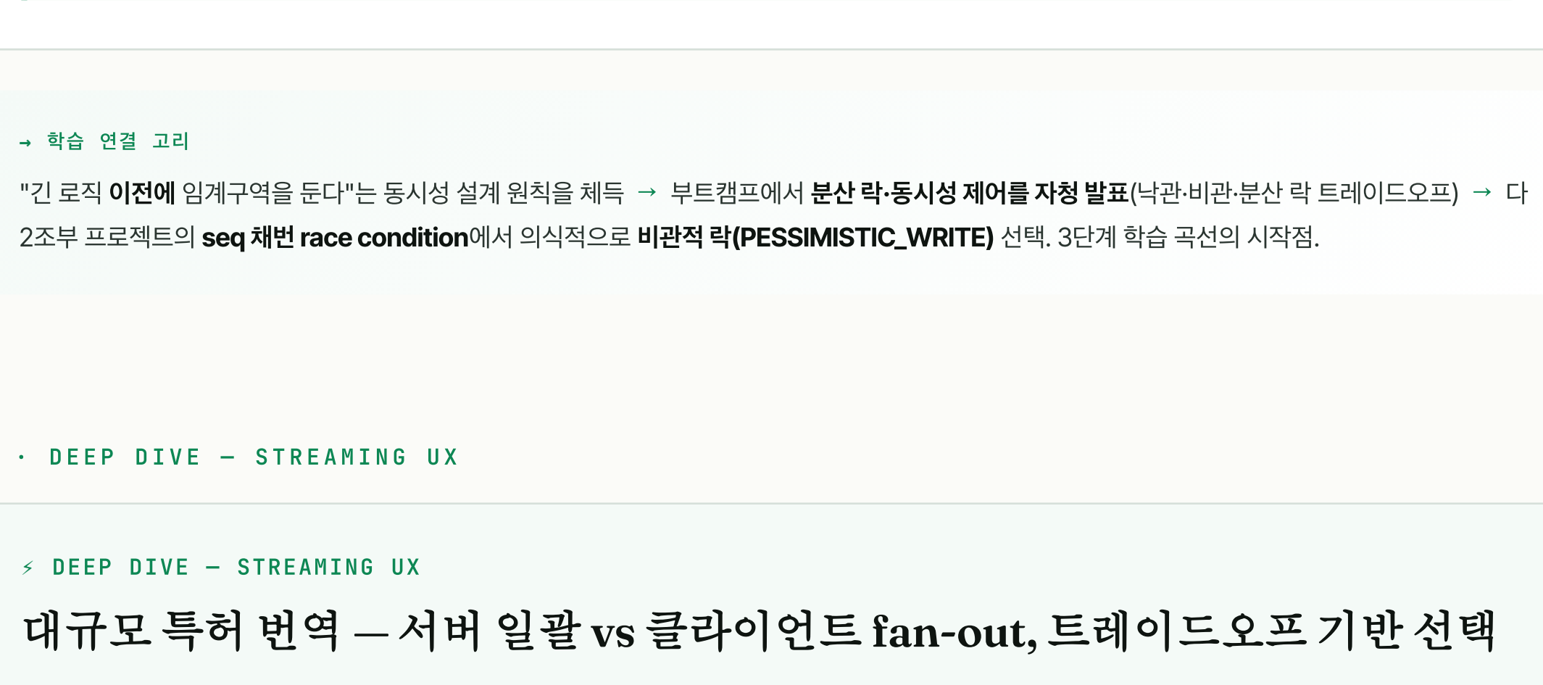
단일 DB 환경에 **오버엔지니어링.** 외부 의존성 추가 + 운영 복잡도 상승. 본 문제는 DB 한 곳에서 풀 수 있음.

CHOSEN · 임계구역 재배치

사전 차감 (Front-loading)

이용량 집계를 **critical section 가장 앞으로 이동**하고 사전 차감 방식 전환. 락 없이도 race 차단 + 실패 시 롤백 가능.

▾ BEFORE / AFTER 구조 비교 보기 클릭해 펼치기



*요점: 긴 시간 소요 로직 **이전에** 임계구역을 두면 락 없이도 race를 막을 수 있다.*
*비동기 모니터링은 호출자 예외 컨텍스트 밖에서 발생하므로 완료 분기에서 **명시적 메트릭·로깅**이 필수.*

R · 동시 다발 요청을 통한 이용량 제한 우회를 **구조적으로 차단**, 비동기 환경 모니터링 누락 제거.

> 📚 당시 한계와 이후 학습
>
> 당시엔 **"여러 방법 중 사전 차감이 맞다"**는 판단으로 해결했으나, 락 기반 대안을 체계적으로 비교하지 못한 상태였음. **이후 학습 결과로 정리한 대안:** 비관적 락(`PESSIMISTIC_WRITE`)으로 차감 row를 잠그거나, 낙관적 락(`@Version`)으로 충돌 시 재시도하는 방식. **해당 환경에서의 판단 근거:** 다운로드 처리 시간이 길어 락 대기 비용이 크므로 사전 차감이 적합. 현재는 환경에 따라 도구를 선택할 수 있는 단계에 도달.

→ 학습 연결 고리

"긴 로직 **이전에** 임계구역을 둔다"는 동시성 설계 원칙을 체득 → 부트캠프에서 **분산 락·동시성 제어를 자청 발표**(낙관·비관·분산 락 트레이드오프) → 다2조부 프로젝트의 **seq 채번 race condition**에서 의식적으로 **비관적 락(PESSIMISTIC_WRITE)** 선택. 3단계 학습 곡선의 시작점.

07 · DEEP DIVE — STREAMING UX

⚡ DEEP DIVE — STREAMING UX

대규모 특허 번역 — 서버 일괄 vs 클라이언트 fan-out, 트레이드오프 기반 선택

두 방식의 비용·체감 응답성 트레이드오프를 정리해 의사결정. 인프라 부하 증가를 감수하고 UX를 택한 근거가 분명한 사례

P · 특허 검색 결과 한 화면에 최대 **200건**의 항목이 노출되며, 각 항목의 제목·요약을 사용자의 언어로 번역해 함께 표시해야 함. 번역 API는 건당 수백 ms 단위라 누적 시간이 큼. 팀 내에서 *"번역 도입 후 로딩 속도 저하"*에 대한 우려가 있었음.

A · 두 가지 방식의 트레이드오프를 비교 후, 인프라 수용량과 UX 중요도를 근거로 방식 B 선택. 프론트와 협업해 응답 처리 흐름 설계.

> **① 방식 A vs B 트레이드오프 정리**
>
> **A**: 서버가 200건 번역 후 한 번에 응답 — 통신은 1회로 효율적이나 사용자는 전부 번역될 때까지 빈 화면 대기. **B**: 원문 즉시 응답 + 클라이언트가 항목별로 번역 fan-out — 서버 통신 1회 → 최대 200회로 증가(**인프라 부하↑**), 대신 첫 화면 대기 시간(TTFB) 극적 단축.
>
> *판단 기준: 어느 쪽이 빠른가보다, 어느 쪽 비용을 감당할 수 있고 어느 쪽 가치가 더 중요한가를 두 축으로 결정.*

> **② B 선택 근거 — 인프라 수용량 + UX 중요도**
>
> 우리 인프라가 200회 동시 요청 수준을 감당 가능하다는 정보가 있었고, 해당 페이지는 사용자 경험이 중요한 페이지로 분류되어 있었음 → **비용은 감당 가능, 가치는 우위**라는 결론으로 B 채택.

> **③ 구현 — 원문 즉시 렌더 + 비동기 도착 순 교체**
>
> 서버는 번역되지 않은 원본을 먼저 빠르게 응답해 화면을 즉시 그림. 이후 클라이언트가 각 항목별로 번역 API를 비동기 호출하고, 응답이 도착하는 순서대로 해당 영역만 원본→번역문으로 교체.
>
> *사용자 경험: 빈 화면 0초 → 원본 즉시 노출 → 번역 도착 항목부터 자연스럽게 갱신.*

R · 팀 내에서 우려가 있던 **"번역 도입 후 로딩 속도 저하"가 눈에 띄는 이슈 없이 출시.** 인프라 부하 증가라는 비용을 사전에 인지·감수한 결정이라 출시 후에도 예상 범위 내에서 동작.

> 📚 당시 한계와 이후 학습
>
> 당시엔 트레이드오프를 직접 비교해 적용했으나, 이것이 일반화된 패턴임은 인식하지 못한 상태였음. **이후 학습 결과로 이해한 패턴: Client fan-out · Progressive Rendering — TTFB(첫 반응 시간) 단축이 체감 응답성을 결정한다**는 원칙이며, **HTTP 스트리밍·SSE·GraphQL @defer** 등 표준이 같은 철학에서 출발. **다음 작업 시 접근법: SSE 같은 표준 프로토콜**이나 **viewport 우선순위 큐**를 함께 검토해 인프라 부하도 같이 줄이는 방향 가능.

08 · STACK

경력 중 사용 스택

영역	스택
Language / Framework	**Java 8** · **Spring 4.x (Legacy)** · JavaScript
Data Access	**MyBatis** · Stored Procedure (T-SQL)
Database	PostgreSQL → **MSSQL 마이그레이션**
Document Parsing	**Apache POI** (MS Office) · **hwplib** (HWP) — E-DRM 시그널 식별
Test	JUnit · Mockito
Frontend (Legacy)	JSP · **AngularJS** · jQuery 계열 — 비동기 렌더링·형광펜 레이어·스크롤·이미지 동기화

위는 **레거시 코드베이스 운영 환경**의 스택. 부트캠프 프로젝트(Java 17 · Spring Boot 3.5 · React/TS 등)에서 **모던 스택으로 학습·적용 병행** → 다음 섹션(GROWTH MAP) 참조.

09 · GROWTH MAP

실무 한계 → 학습 동기 → 부트캠프 적용

실무에서 시행착오로 풀어낸 문제들이 그대로 **"왜 그게 효과 있었는지 알고 싶다"**는 공부로 이어졌습니다. 이론으로 채운 뒤 다시 프로젝트에서 검증하는 사이클이 두 번 완결.

🔄 GROWTH CYCLE #1 — SQL · 벤더 종속성 · DB 깊이

PG → MSSQL 슬로우 쿼리 시행착오 → ORM 학습 + 실행 계획 공부 → 부트캠프 프로젝트 적용

실무 · 100% MyBatis 환경에서 슬로우 쿼리를 **경험적 시행착오로 풀었지만**, "왜 효과 있는지" 설명할 수 없는 부분이 마음에 남음.

학습 · 영속성 컨텍스트·JPQL·벤더 독립 데이터 접근 계층 설계를 학습. 옵티마이저·실행 계획·카디널리티 추정 개념도 함께 정리. **"왜 ORM이 필요한지"를 실무 경험으로 설명 가능.**

적용 · 부트캠프 프로젝트에서 **JPA + QueryDSL**로 데이터 액세스 계층 직접 설계. PostGIS 같은 벤더 특수 기능은 Native Query로, 일반 도메인은 JPA로 — 이 분기 기준도 실무에서 체감한 한계로부터 나온 판단.

🔄 GROWTH CYCLE #2 — 동시성 제어

비동기 다운로드 race 직감으로 해결 → 락 트레이드오프 발표 → 다2조부 비관적 락 의식적 선택

실무 · "긴 로직 이전에 임계구역을 둔다"는 방식으로 race를 풀었지만, **어떤 락이 어떤 상황에 맞는지 체계적으로 모름.**

학습 · 부트캠프 스터디에서 **낙관적/비관적/분산 락 트레이드오프**를 자청 발표. 각 락의 동작 원리·임계구역 크기·충돌 빈도에 따른 선택 기준을 코드 레벨로 검증.

적용 · 다2조부 프로젝트의 **seq 채번 race condition**에 비관적 락(`PESSIMISTIC_WRITE`) 적용. 분산 락도 검토 후 "단일 인스턴스 토폴로지엔 오버엔지니어링"이라는 판단으로 배제 — **여러 옵션을 알고 의도적으로 선택**한 사례.

← 포트폴리오 메인으로

▶ 라이브 데모 (평일 09-21시)

danbeekimm/da2joburureung

01 · OVERVIEW

정합성 보장과 장애 격리를 설계한 B2B 물류 MSA

직전 모놀리식(omin)에서 한 도메인에 결함이 나면 전체가 멈추고, 특정 도메인 전용 기술(PostGIS)을 전 팀원이 설치해야 했던 비효율을 직접 겪고 **도메인별 독립 배포·기술 선택이 가능한 MSA**로 확장. B2B 도메인을 코드에 녹이기 위해 **DDD 4계층(헥사고날)**을 적용하고, 서비스 간 통신은 "**검증=동기(Feign), 통지=비동기(Kafka)**"로 의도적으로 분리했습니다.

> **"분산 트랜잭션 없이 정합성을, 장애 전파 없이 통신을."**
>
> 전국 17개 허브를 거점으로 한 **스파르타 물류** — 업체·허브·상품·주문·배송을 MSA로 분리한 B2B 물류 시스템에서 **company-service 전담 + DeliveryManager 도메인** 설계. Outbox 패턴 · Circuit Breaker · OR-Tools VRPTW · 비관적 락을 비즈니스 근거에서 출발해 채택.
>
> - 2026 · 부트캠프 5인
> - 담당: company-service + delivery-service 내 3개 도메인
> - Spring Cloud 2025 · Kafka · OR-Tools · Keycloak

at-least-once
이벤트 정합성 보장
Outbox 패턴 · 분산 트랜잭션 없이

5분 → 3초
배송 배정 자동화
수동 배정 → OR-Tools VRPTW (100건 기준 · 데모)

DB-level
race 원천 차단
비관적 락 · 재시도 로직 없이

02 · SERVICE OVERVIEW

스파르타 물류 — MSA 기반 국내 B2B 물류 관리 및 배송 시스템

*전국 **17개 지역 허브 센터**를 거점으로 한, **"공급업체 → 허브 → 허브 → 수령업체"** 전 구간을 자동화한 **B2B** 물류 관리 플랫폼.*

스파르타 물류는 전국 17개 도/광역시에 허브 센터를 보유한 가상의 B2B 물류 회사입니다. 각 허브는 여러 업체의 물건을 보관하며, 주문이 발생하면 시스템은 **출발 허브 → 목적지 허브 간 화물 이동(허브 배송 담당자)**과 **최종 허브 → 수령 업체 배송(업체 배송 담당자)** 두 트랙을 분리해 처리합니다.

예시 시나리오 — "부산 수산물 도매 업체가 경기도 일산의 건조 식품 가공품 50개를 주문" → 시스템이 **경기도 허브 → 부산 허브**로 화물 이동을 계획하고, 도착하면 **부산 허브 소속 업체 배송 담당자**가 최종 업체로 전달.

시스템 핵심 도메인 (8개)

01 · COMPANY ★ 담당
업체 관리 (생산 · 수령)
생산업체·수령업체로 구분, 모든 업체는 특정 허브에 소속. 업체 삭제 시 주문/상품/배송 등 타 서비스에 이벤트 통지 필요.

02 · HUB · HUBPATH
17개 허브 · 허브 간 경로
전국 도/광역시 17개 센터 관리 + 허브 간 이동 경로 매핑. 모든 화물의 시작/도착점.

03 · PRODUCT
상품 · 재고 관리
모든 상품은 특정 업체와 허브에 소속. 주문 발생 시 재고 감소, 취소 시 복원.

04 · ORDER
주문 라이프사이클
주문 생성 → 재고 검증·차감 → 배송 트리거. 재고 부족 시 주문 실패. 삭제 시 논리 삭제 (deleted_at).

05 · DELIVERY ★ 담당
배송 · 담당자 · 경로 (3개 도메인 통합)
배송 본체 + DeliveryManager(허브당 10명 + 전체 10명) + DeliveryRoute(경로 기록)를 한 컨텍스트로 통합. **매일 06:00 VRPTW 자동 배정.**

06 · USER · AUTH
4단계 권한 관리
마스터 / 허브 관리자 / 배송 담당자 / 업체 담당자. **Keycloak + JWT**로 권한 검증.

07 · NOTIFICATION
Slack 메시지 관리
시스템·사용자 발신 슬랙 메시지 전부 엔티티에 저장. Incoming Webhook으로 외부 발송.

08 · AI
AI 상품 정보 생성
상품 등록 시 LLM 기반 설명 자동 생성. AI 응답 로그는 별도 트랜잭션(REQUIRES_NEW)으로 분리해 본 로직 무영향.

2-Track 배송 구조 — 발제문 기반

트랙	역할	인원	배정 방식
허브 배송 담당자	허브 간 화물 이동 (경기도 허브 → 부산 허브)	전체 10명	대기시간·순번 기반 **라운드로빈**
업체 배송 담당자	최종 허브 → 수령 업체 배송	허브당 10명	**OR-Tools VRPTW** (시간창·차량 부하 균등)

두 트랙의 **업무는 명확히 분리** — 허브 배송 담당자는 허브 → 업체 배송 불가, 업체 배송 담당자는 허브 간 배송 불가. 이 도메인 특성을 코드로 강제하기 위해 **배정 알고리즘 자체를 트랙별로 분리**(라운드로빈 / VRPTW)했습니다.

03 · DESIGN GOALS

설계 도전 과제와 핵심 기여

핵심 도전 과제

- 분산 트랜잭션 없이 **이벤트 유실 · 유령 이벤트** 차단 — DB 커밋과 Kafka 발행의 원자성 불가
- 외부 서비스 장애 시 **연쇄 전파(cascading failure)** 차단 — 대기 스레드 누적 문제
- B2B의 **시간창 SLA** 준수 — 후처리 정렬 아닌 제약 조건으로 모델링 필요
- seq 채번의 **Read-Modify-Write race** — 동시 등록 시 중복 발급 위험

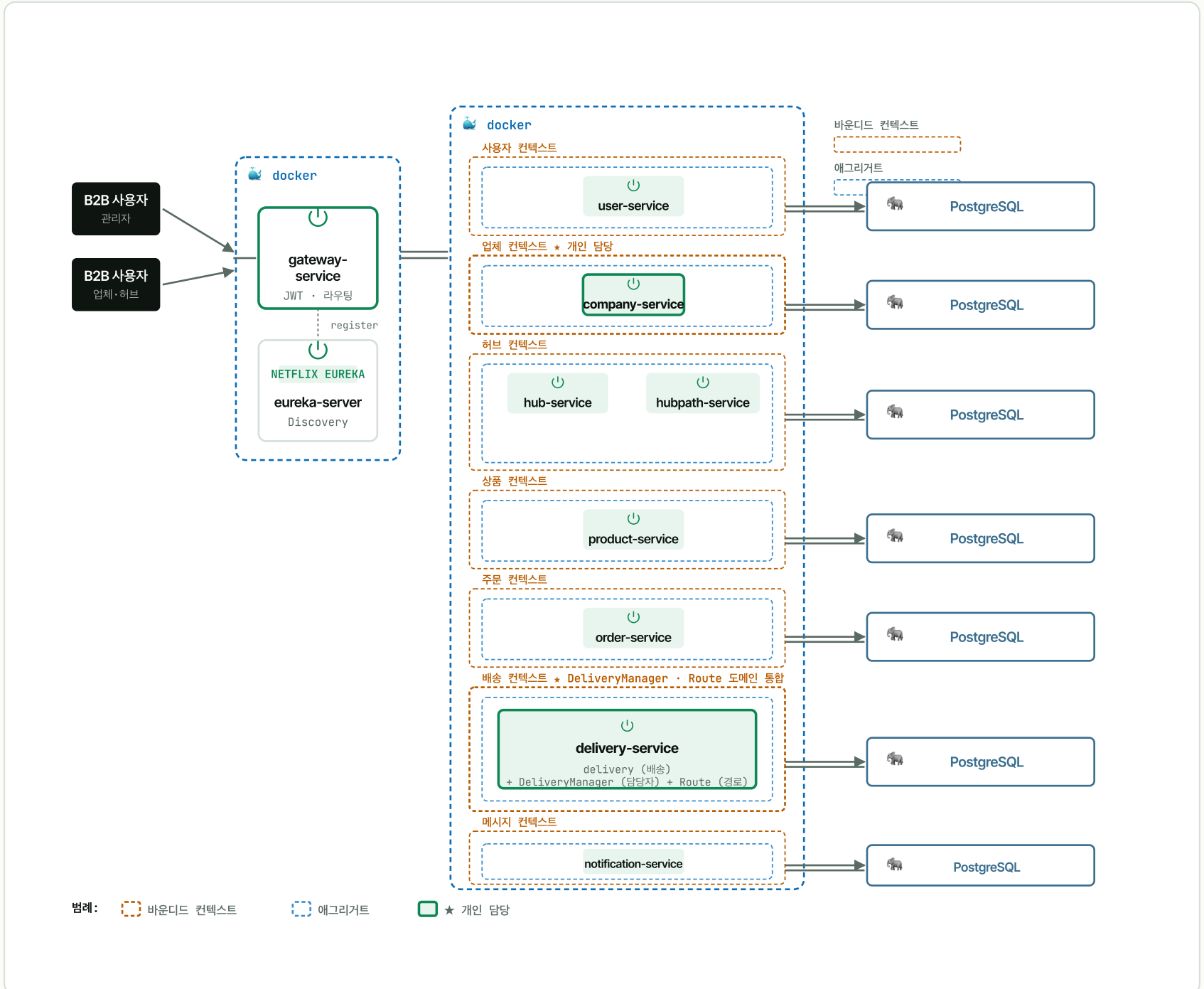
핵심 기여 (개인)

- company-service 전담 + DeliveryManager 도메인 **DDD 4계층 헥사고날** 설계
- 업체 삭제 이벤트의 **Kafka + Outbox 패턴** 구현 (Saga 대안 검토 후 채택)
- user/hub/order 호출의 **OpenFeign + Resilience4j CB** 적용
- 매일 06:00 cron의 **OR-Tools VRPTW** 일괄 배송 최적화
- seq 채번 **비관적 락(PESSIMISTIC_WRITE)** — 스터디에서 학습한 트레이드오프 적용

04 · ARCHITECTURE

시스템 아키텍처

12개의 마이크로서비스를 **Spring Cloud Gateway · Eureka**로 묶고, 통신은 **검증=동기(Feign), 통지=비동기(Kafka Outbox)**로 분리합니다. 바운디드 컨텍스트별 PostgreSQL 분리(database-per-service)로 서비스 독립성·장애 격리를 확보했습니다.



*설계 의도: 12개 서비스를 7개 바운디드 컨텍스트(파선)로 묶고, 각 컨텍스트는 자기 PostgreSQL을 가짐(database-per-service). 배송 컨텍스트는 처음엔 `delivery / delivery_manager / delivery_route` 3개 서비스로 분할 시도했으나, **강한 응집성·동일 트랜잭션 경계 요구** 때문에 `delivery-service` 한 컨텍스트로 통합하고 내부 도메인으로 분리.*

설계 의사결정과 근거

결정	대안	선택 이유 — 트레이드오프 명시
MSA + DDD 헥사고날	모놀리식 (omin)	직전 omin에서 한 도메인에 결함이 나면 전체가 멈추고 도메인 전용 기술(PostGIS)을 전 팀원이 설치해야 했던 비효율 직접 경험 → **도메인별 독립 배포·기술 선택**이 가능한 MSA로.
database-per-service	공유 DB · schema 분리	바운디드 컨텍스트별 PostgreSQL 분리 → 서비스 독립성·장애 격리. **데이터 정합성은 Outbox 이벤트로 별도 보장.**
"검증=동기, 통지=비동기" 통신 분리	전부 동기 / 전부 비동기	정합성 검증("이 hub가 있나?")은 응답을 받아야 진행 가능 → 동기(Feign). 삭제 통지는 응답 불필요 → 비동기(Kafka). **의도적 분리.**
Outbox 패턴	Saga · 2PC · 직접 Kafka 발행	단방향 통지·응답 불필요라 full Saga는 과함. 2PC는 분산 트랜잭션 회피. 직접 발행은 유실/유령 이벤트 위험 → **at-least-once + 결과적 일관성**의 Outbox가 적정.
단일 인스턴스 → 분산 락 생략	ShedLock · 리더 선출	각 서비스 단일 인스턴스 배포 토폴로지 → Outbox 스케줄러 중복 실행 자체가 발생 X. **수평 확장 시** ShedLock + 컨슈머 멱등 처리 추가 예정.

05 · KEY CONTRIBUTION ①

CONTRIBUTION #1

이벤트 유실·유령 이벤트 없는 비동기 통지 — Kafka + Outbox 패턴

분산 트랜잭션 없이 at-least-once · 결과적 일관성 보장. 단위 테스트로 단일 이벤트 실패가 나머지 발행을 막지 않음 검증

P · 업체 삭제 시 주문·상품 등 타 서비스 통지가 필요한데, "DB 커밋 → Kafka 발행" 사이 발행 실패 시 **이벤트 영구 유실**(DB는 지워졌는데 남들은 모름), 발행 후 DB 롤백 시 **유령 이벤트**(다른 서비스는 삭제됐다고 알고 처리). 분산 트랜잭션(2PC)은 가용성을 해치고 MSA와 부적합.

A · 삭제 트랜잭션 내 `OutboxEvent` (PENDING) **원자 저장** → DB 커밋과 함께 커밋(또는 함께 롤백). `OutboxEventScheduler` 가 `@Scheduled(fixedDelay=10000)` 로 10초마다 PENDING을 Kafka로 발행→ `PUBLISHED` . 실패 시 PENDING 유지 → 다음 주기 자동 재발행.

R · 분산 트랜잭션 없이 **at-least-once 발행 + 결과적 일관성** 보장. 단위 테스트로 단일 이벤트 실패가 나머지 발행을 막지 않음 검증.

CODE `CompanyService.java · deleteCompany 원자 저장` `OutboxEventScheduler.java · 10s 폴링 발행` `OutboxEvent.java · 상태 모델` `OutboxEventSchedulerTest.java · 단위 테스트`

대안 검토 — 옵션별 트레이드오프 · 트랜잭셔널 Outbox 채택 클릭해 펼치기

REJECTED · 2PC
분산 트랜잭션 코디네이터
모든 참여자가 잠금을 유지 → **가용성 저하**, MSA와 부적합. 코디네이터 자체가 SPOF.

REJECTED · Saga
보상 트랜잭션 체인
다중 단계 트랜잭션엔 강력하나, 본 케이스는 **단방향 통지·응답 불필요**라 full Saga는 과함. 보상 로직 작성 비용도 큼.

CHOSEN · OUTBOX
트랜잭셔널 Outbox
도메인 변경과 이벤트를 **같은 트랜잭션에 기록** → 별도 publisher가 polling으로 Kafka 발행 → 실패 시 PENDING 유지·재시도. **at-least-once**.

구현 코드 — Outbox 원자 저장 (Java) 클릭해 펼치기

```java
// 업체 삭제 트랜잭션 내 Outbox 원자 저장
@Transactional
public void deleteCompany(UUID companyId) {
    companyRepository.delete(companyId);
    outboxRepository.save(new OutboxEvent(
        "company.deleted", companyId, payload, "PENDING"
    ));  // 같은 트랜잭션 — 원자 보장
}

// 10초마다 PENDING → Kafka
@Scheduled(fixedDelay = 10000)
public void publish() {
    outboxRepository.findByStatus("PENDING").forEach(event -> {
        kafkaTemplate.send(event).addCallback(
            ok -> outboxRepository.markPublished(event.getId()),
            fail -> log.warn("retry next tick")
        );
    });
}
```

06 · KEY CONTRIBUTION ②

CONTRIBUTION #2

검증=동기, 통지=비동기 — Feign + Circuit Breaker로 통신 의도 분리

외부 서비스 장애의 연쇄 전파 차단 — CB(window 10·실패율 50%·OPEN 10s·slow-call 3s) + Retry 3회로 fallback 즉시 반환

P · 외부 서비스 호출이 타임아웃까지 블로킹되면 **대기 스레드 누적 → 내 서비스까지 연쇄로 죽음**(cascading failure). 단순 동기 호출은 한 서비스 장애가 전체로 번지는 구조.

A · 통신 의도를 의도적으로 분리: **"정합성 검증=동기(Feign)", "삭제 통지=비동기(Kafka)"**. user/hub/order 호출은 `@CircuitBreaker` + `@Retry` 로 감쌈. CB(window 10·실패율 50%·OPEN 10s·slow-call 3s·retry 3회/500ms)로 fallback 즉시 반환.

R · CB OPEN 시 즉시 fallback 반환 → 대기 스레드 누적·연쇄 장애 차단. CB 상태전이(OPEN/CLOSED/HALF_OPEN)·호출차단·slow-call 전체 로깅.

CODE `HubClientImpl.java · CB+fallback` `ResilienceConfig.java · 상태전이 로깅` `application-docker.yml · CB 설정값`

연쇄 전파가 어떻게 번지나 (Why → Why)

실제 장애 시나리오 — hub-service 일시 장애

- 외부 호출이 **타임아웃까지 스레드를 점유**
- 풀에서 스레드가 빠르게 소진 → 새 요청이 대기 큐에 적체
- 대기 요청도 결국 타임아웃 → company-service 자체 응답 지연
- fail-fast가 없으면 장애가 **양방향으로 전파** → CB OPEN 시 즉시 fallback해 점유 시간을 타임아웃에서 수 ms로 끊어냄

구현 코드 — Circuit Breaker / Retry (Java) 클릭해 펼치기

```java
@Retry(name = "hubService")
@CircuitBreaker(name = "hubService", fallbackMethod = "hubServiceFallback")
public boolean validateHubExists(UUID hubId) {
    return hubClient.exists(hubId);  // 동기 검증
}

private boolean hubServiceFallback(UUID hubId, Throwable t) {
    log.warn("Hub validation fallback — CB OPEN. hubId={}", hubId);
    return false;  // 즉시 반환 → 스레드 점유 X
}
```

설정 항목	값	이 값으로 정한 이유
sliding window	`10 calls`	실패율 산출 표본 크기. 너무 작으면 1~2건 실패에 과민 반응, 너무 크면 장애 감지가 늦음 → 둘의 절충점
failure rate threshold	`50%`	retry 3회를 통과하고도 절반이 깨지면 일시적 결함이 아니라 명백한 장애로 판단. 그 이하(예: 30%)면 단발성 오류에도 차단돼 과민
wait in OPEN	`10s`	외부 서비스 재기동·순간 부하 해소에 통상 수~십수 초. 더 짧으면 복구 전 재요청이 몰려 재차 OPEN, 더 길면 정상화 후에도 차단 지속
half-open calls	`3`	1건 성공은 우연일 수 있어 성급한 CLOSE를 방지. 3건 시험 호출로 실제 복구 여부를 확인 후 닫음
slow call threshold	`3s`	등록 플로우가 허용할 수 있는 지연 상한. Feign read timeout보다 짧게 둬 '느린 성공'도 장애로 간주해 미리 차단
retry	`3회 / 500ms`	패킷 손실·순간 GC 같은 일시 결함은 보통 1~2회 재시도로 흡수. 500ms 간격으로 순간 부하 분산, 3회 초과 시 일시 결함이 아니라 보고 CB에 위임

값 선정 전제 — 데모 트래픽 기준의 보수적 baseline. Resilience4j 기본값을 출발점으로 위 근거에 따라 조정했고, **운영 전환 시 실제 호출량·지연 분포로 부하 테스트 후 재튜닝**하는 것을 전제로 한 초기값.

07 · KEY CONTRIBUTION ③

CONTRIBUTION #3

시간창을 제약으로 모델링한 일괄 배송 최적화 — OR-Tools VRPTW

B2B SLA의 관건인 시간창 준수를 사후 정렬이 아니라 솔버의 직접 제약으로 모델링. 수동 5분 → 자동 3초 (100건 기준 · 데모)

P · B2B 물류는 업체 영업시간(예: "오전 9~11시 입고")이 **SLA의 핵심**. 어기면 계약 위반 → **시간창을 제약(constraint — 솔버가 반드시 지켜야 하는 조건)으로 모델링**해야지, 후처리 정렬로는 보장 불가. 단순 라운드로빈(들어온 순서대로 담당자에게 균등 배분)은 거리·시간창·차량 부하 균등을 못 고려해 비효율.

A · 매일 06:00 스케줄러가 허브별 당일 배송을 모아 **VRPTW로 일괄 최적화**. 트랙 분리 — **허브 간 = 라운드로빈, 업체 = OR-Tools VRPTW**. 제약을 모두 만족하는 해가 없으면(infeasible) 시간창을 단계적으로 완화해 재시도.

> **입력** — Haversine 공식(위·경도 좌표로 두 지점 간 직선거리 계산)으로 시간·거리 행렬 생성
>
> **제약** — 시간창 ±30분 · 라우트 최대 480분(8시간) · 차량당 상한 `ceil(총건/담당자수)` 로 부하 균등
>
> **솔버** — 가까운 지점부터 이어 붙여 초기 경로를 빠르게 만드는 휴리스틱(`PATH_CHEAPEST_ARC`) + 그 경로를 반복 개선하며 부분 최적에 갇히지 않도록 탐색하는 기법(`GUIDED_LOCAL_SEARCH`), 탐색 5초 제한
>
> **가용성** — 해가 없을 때(infeasible) 시간창 단계적 완화 + 스케줄러 레벨 3회 재시도(10초 간격)

R · 시간창을 솔버의 직접 제약으로 모델링해 **SLA 보장 가능한 자동 배정 체계** 확보. 해가 없을 때도 시간창 완화로 가용성 확보.

CODE `OrToolsRouteOptimizationService.java · 솔버 구성` `CompanyDeliveryAssignmentService.java · 시간창 완화` `CompanyDeliveryAssignmentScheduler.java · 06:00 cron+재시도` `HubDeliveryAssignmentService.java · 라운드로빈`

대안 검토 — 왜 OR-Tools인가 · 제약 만족 솔버라 채택 클릭해 펼치기

REJECTED · K-means
거리 기반 군집화
시간창·차량용량을 **제약으로 못 넣음**. 클러스터링 후 시간창 검증은 사후 필터링 → 위반 건은 다음 날로 밀려 **SLA 위반**. 초기값에 의존하는 휴리스틱(경험적 어림법)이라 결과도 불안정.

REJECTED · pgRouting
경로 탐색 (A→B 최단)
경로 탐색엔 강하지만 **"어떤 묶음을 누구에게 배정"하는 할당(assignment) 문제는 안 풂**. 지리 공간 시각화·근사 계산용에 가깝고, 제약 만족 문제(CSP — 여러 제약을 동시에 만족하는 해를 찾는 문제)에는 부적합.

CHOSEN · OR-TOOLS
제약 만족 솔버 (CSP)
시간창·차량 용량·작업 시간을 **솔버가 직접 지키는 제약으로 선언** → 솔버가 제약 안에서 최적해 탐색. 해가 없으면 시간창 완화 단계적 재시도 가능.

OR-Tools VRPTW 구성요소 상세 (Depot · Dimension · Time Window · 솔버 파라미터) 클릭해 펼치기

개념	코드 표현	의미
Depot	`depot` 인덱스	모든 라우트의 출발·복귀 지점(허브)
Dimension	`AddDimension(...)`	라우트 따라 누적되는 값(시간·거리·용량)에 제약 부여
Time Window	Dimension slack + 노드별 setRange	업체별 영업시간 — 도착 가능 구간
1st Solution Strategy	`PATH_CHEAPEST_ARC`	초기해를 빠르게 만드는 휴리스틱 (가까운 노드 우선)
Local Search Metaheuristic	`GUIDED_LOCAL_SEARCH`	초기해를 점진적으로 개선 — 국소최적 탈출 메커니즘
Time Limit	`setTimeLimit`	솔버 탐색 시간 상한 — 충분히 좋은 해에서 stop

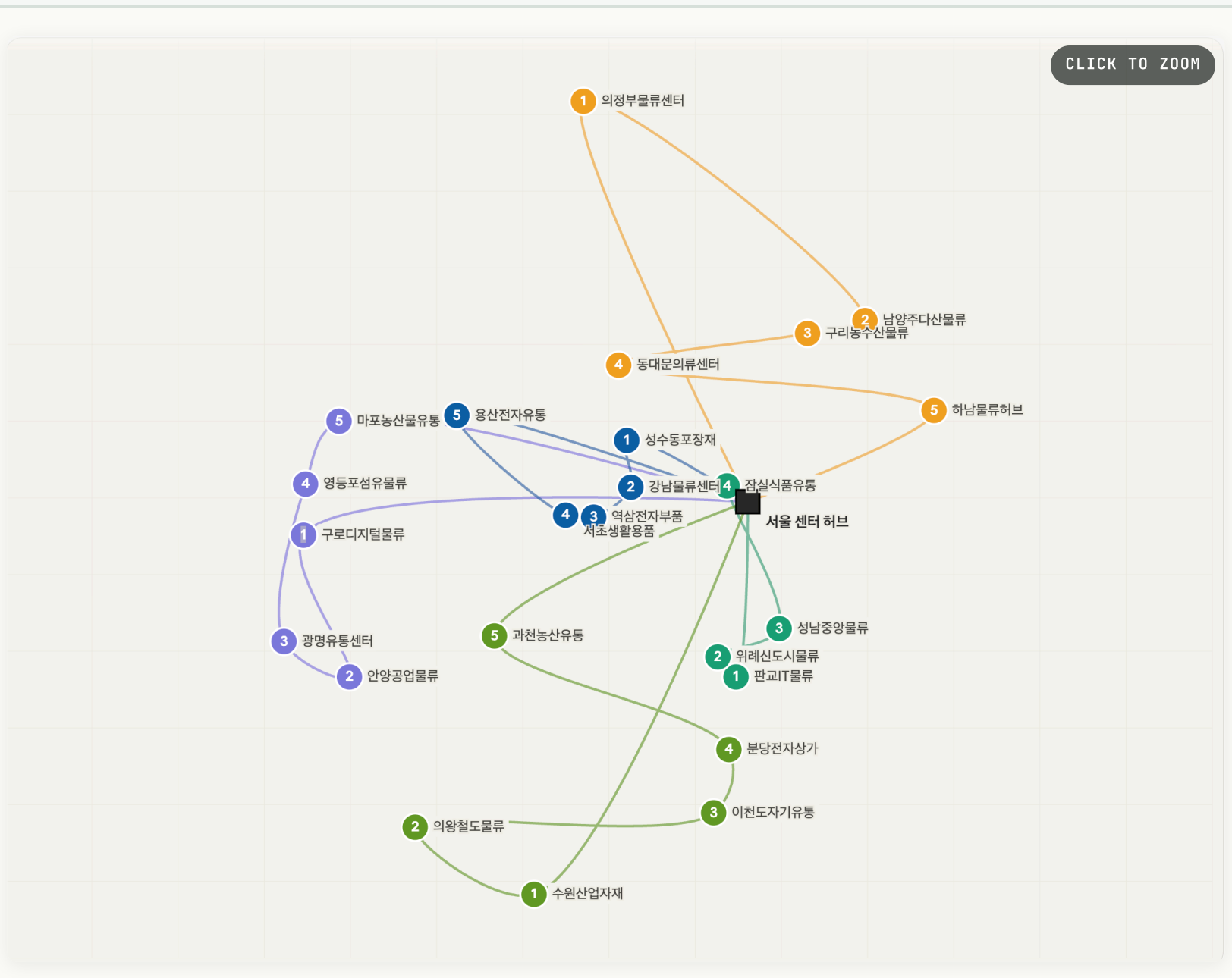
구현 코드 — VRPTW 핵심 구성 (Java) 클릭해 펼치기

```java
// VRPTW 핵심 구성
// nodes: 방문 지점 수(허브+업체) / vehicles: 담당자 수 / depot: 출발·복귀 지점(허브) 인덱스
RoutingIndexManager manager = new RoutingIndexManager(
    nodes, vehicles, depot
);

// AddDimension: 시간/거리/용량처럼 라우트 따라 누적되는 값에 제약 부여
routing.AddDimension(
    timeCallback,
    30,        // 노드별 대기 가능 여유(slack) — 시간창 도착에 30분 버퍼
    480,       // 한 담당자 라우트 총 시간 상한(분) = 8시간
    false, "Time"  // depot 시작 시점에 누적값 리셋 안 함
);

// 차량당 상한 — 부하 균등
int maxPerVehicle = (int) Math.ceil((double) total / vehicles);

// 솔버 구성 = "초기해를 빠르게 + 그 해를 개선" 2단
RoutingSearchParameters params = main.defaultRoutingSearchParameters()
    .toBuilder()
    .setFirstSolutionStrategy(FirstSolutionStrategy.PATH_CHEAPEST_ARC)          // 가까운 노드부터 그리디로 초기해
    .setLocalSearchMetaheuristic(LocalSearchMetaheuristic.GUIDED_LOCAL_SEARCH) // 초기해를 개선하며 국소최적 탈출
    .setTimeLimit(Duration.ofSeconds(5))
    .build();
```

왜 일괄(batch)인가 — 실시간 배정 = 들어오는 순서대로 그리디 → **국소 최적**(지금 가까운 사람에게 줬더니 나중 배송이 꼬임). VRPTW는 **당일 전체를 한 번에 모아 전역 최적**. B2B는 "즉시"보다 "당일 정시 배송"이 중요 → 일괄이 비즈니스에 더 맞음.

해가 없을 때(infeasible) — 시간창만 단계적 완화 · 특정 일자에 시간창이 좁고 물량이 몰리면 솔버가 **infeasible**을 반환한다. 단순 예외 throw는 가용성을 해치므로, 다른 제약은 두고 **시간창만 ±30 → ±60 → ±90분**으로 단계적으로 풀어 재시도한다. *차량 추가 투입은 인력 자원 문제라 시스템 제어 밖, 모든 제약 무시 후 재계산은 SLA 약속 자체가 깨지므로 배제* — SLA 영향이 가장 작은 시간창부터 양보했다. 모두 실패하면 `ROUTE_OPTIMIZATION_FAILED`로 운영자에게 알리고, 스케줄러 레벨에서 10초 간격 3회 재시도한다.

실제 OR-Tools VRPTW 솔버 출력 — 담당자별 경로 시각화 VRPTW · 실측



실제 VRPTW 솔버 결과를 지도에 매핑한 경로 시각화. 각 색상이 한 담당자의 경로이며, 숫자는 방문 순서.

- ✓ **5명 담당자별 색상 구분** — 같은 색끼리 묶인 노드가 한 담당자의 하루 동선
- ✓ **방문 순서 표시**(① ② ③ …) — 솔버가 결정한 실제 순회 경로(sequence)
- ✓ **허브 출발 → 업체 순회 → 허브 복귀** — VRP의 전형적 라우트 구조가 그대로 출력에 반영

08 · KEY CONTRIBUTION ④

CONTRIBUTION #4

짧고 잦은 임계구역엔 비관적 락 — seq 채번 race condition 해결

경력의 동시성 학습 → 스터디 발표 → 본 프로젝트 적용. 3단계 학습 곡선의 매듭

P · 배송담당자 등록 시 "최대 seq 조회 → +1 저장" 사이 동시 요청이 끼면 **Read-Modify-Write race** → 같은 seq가 두 번 발급.

A · `@Lock(LockModeType.PESSIMISTIC_WRITE)` 로 최대 seq row를 select for update. *대안 검토: 낙관적 락은 어차피 매번 충돌하는 채번엔 재시도 비용만 누적, 분산 락은 단일 인스턴스 토폴로지엔 과함* → **"충돌 잦고 짧은 임계구역"엔 비관적 락이 최적**이라 판단.

R · 동시 등록 요청에서도 seq 유일성 보장. 재시도 로직 없이 **DB 레벨에서 차단**. 추가로 **락 타임아웃 3초**를 명시해 장기 트랜잭션·데드락 시 `LockTimeoutException` 으로 빠른 실패 처리 — 무한 대기로 인한 스레드 점유 차단. **동시 등록 시나리오 단위 테스트**로 seq 유일성을 검증했다.

CODE `JpaDeliveryManagerRepository.java · @Lock+timeout 3000`

대안 검토 — 비관적 락 · 짧고 잦은 임계구역에 채택 클릭해 펼치기

REJECTED · 낙관적 락
버전 충돌 → 재시도
채번은 **매번 같은 row를 만짐** → 거의 항상 버전 충돌 발생 → 재시도 비용만 누적되고 처리량 저하.

REJECTED · 분산 락
Redis · ZooKeeper
단일 인스턴스 배포 토폴로지엔 **오버엔지니어링**. 외부 의존성도 늘어남.

CHOSEN · 비관적 락
SELECT ... FOR UPDATE
임계구역이 **짧고 충돌이 잦음** → 락 대기 부담은 작고 충돌 차단 효과는 큼. 재시도 로직 없이 DB 레벨에서 차단.

구현 코드 — 비관적 락 seq 채번 (Java) 클릭해 펼치기

```java
@Lock(LockModeType.PESSIMISTIC_WRITE)
@QueryHints({@QueryHint(name = "jakarta.persistence.lock.timeout", value = "3000")})
@Query("select dm from DeliveryManager dm "
     + "where dm.hubId = :hubId order by dm.seq desc")
Optional<DeliveryManager> findTopByHubIdOrderBySeqDesc(UUID hubId);
// → SELECT ... FOR UPDATE — 동시 요청은 대기 후 순차 처리, 3초 초과 시 LockTimeoutException
```

3단계 학습 곡선 연결 — 전 회사의 **비동기 다운로드 race condition**을 해결하면서 "정확히 어떤 락이 어느 상황에 맞는지" 몰랐던 게 학습 동기. 부트캠프 스터디에서 **낙관적/비관적/분산 락 트레이드오프**를 자청 발표. 본 프로젝트에서 **"충돌 잦고 짧은 임계구역엔 비관적 락"**을 실제로 적용 — 한 사이클 완결.

09 · DEPLOYMENT · INFRA

배포 인프라 — IaC 한 번으로 AWS + Cloudflare 전 구간 프로비저닝

16개 컨테이너 규모의 MSA를 전시·데모용으로 **월 ~$14**에 운용하기 위해, 인프라 전체를 **Terraform 단일 코드베이스(IaC)**로 선언하고 **GitHub Actions + OIDC**로 빌드·배포를 자동화했습니다. "**비용 · 보안 · 운영부담**"을 판단 기준으로, 매니지드 오케스트레이션 대신 단일 EC2 + Cloudflare Tunnel 조합을 의도적으로 선택했습니다.

월 ~$14
비용 최적화
ALB/EIP 제거 · 평일 09~21시만 자동 가동 (24/7 대비 ~64%↓)

keyless
OIDC 기반 CI/CD
장기 액세스키 0개 · 임시 자격증명으로 9개 이미지 push

1× apply
IaC 단일 적용
VPC·EC2·RDS·ECR·IAM·Scheduler + Cloudflare까지 선언적 관리

CONTRIBUTION #5

Terraform IaC + OIDC 키리스 CI/CD + Cloudflare Tunnel 무포트 노출

매니지드 과투자 대신 "전시 목적에 맞는 최소 비용 · 최소 공격면" 아키텍처를 직접 설계·구축

CI · GitHub Actions — `develop` push 시 9개 서비스를 **matrix 병렬 빌드** → ECR push(`:latest` , `:sha`). **OIDC**로 AWS 임시 자격증명을 발급받아 **장기 액세스키를 저장하지 않음.** `type=gha` 레이어 캐시로 빌드 시간 단축.

IaC · Terraform — VPC(NAT 없는 퍼블릭 서브넷)·EC2·RDS(프리티어)·ECR·IAM(OIDC role)·EventBridge Scheduler·Lambda를 하나의 코드로 선언. **Cloudflare(Tunnel·DNS·SSL)까지 같은** `apply` 로 생성하고, 터널 토큰을 EC2 user_data에 자동 주입.

네트워킹 · Cloudflare — ALB/EIP 없이 **Tunnel로 오리진을 노출**해 인바운드 포트를 전부 닫고(공격면↓), 무료 Universal SSL로 HTTPS 종단. 운영시간 외/주말엔 **Worker가 점검 페이지로 폴백**.

비용 운영 · EventBridge + Lambda — 평일 **08:40 start / 21:00 stop**으로 EC2·RDS를 자동 전원 제어, 전시 시간대만 가동해 비용을 절반 이하로 절감.

CODE `build-push-ecr.yml · OIDC matrix×9` `iam.tf · OIDC role 신뢰정책` `user_data.sh.tftpl · EC2 부트스트랩` `cloudflare.tf · Tunnel·DNS·SSL` `scheduler.tf · 평일 자동 전원`

비용·보안 의사결정 — 대안별 트레이드오프 · EC2 + CF Tunnel 채택 클릭해 펼치기

REJECTED · ECS / EKS
매니지드 오케스트레이션
전시 규모엔 **과투자**. 기존 docker-compose를 그대로 재사용하는 편이 훨씬 저렴하고 단순.

REJECTED · ALB + EIP
표준 인바운드 노출
ALB 월 ~$22 + EIP 미사용 요금. 인바운드 포트를 열어야 해 **공격면 증가**.

CHOSEN · EC2 + CF Tunnel
단일 박스 + 터널
인바운드 포트 **0개**로 보안↑, ALB/EIP 비용 0, 무료 SSL. 전시 목적의 비용·보안 균형에 최적.

배포 파이프라인 · 런타임 토폴로지 (다이어그램) 클릭해 펼치기

```
# 배포 파이프라인 & 런타임 토폴로지
push(develop) ─▶ GitHub Actions (OIDC · matrix ×9) ─▶ Amazon ECR
                                                       │ pull
사용자 ─https─▶ Cloudflare (Universal SSL + Worker 폴백)       ▼
       └Tunnel(무포트)─▶ EC2 t3.medium (앱9 + kafka/redis + cloudflared) ─▶ RDS (DB 8개)
                           ▲ EventBridge + Lambda — 평일 08:40 start / 21:00 stop
```

10 · STACK

기술 스택

영역	스택
Backend	Java 17 · Spring Boot 3.5.12 · **Spring Cloud 2025.0.1** (Eureka · Gateway · OpenFeign)
Messaging	Spring Kafka · **Outbox 패턴**
Resilience	Resilience4j · **Circuit Breaker · Retry**
Data	PostgreSQL (database-per-service) · Redis · JPA · QueryDSL
Optimization	**Google OR-Tools 9.12.4544 (VRPTW)**
Infra (IaC)	**Terraform** · AWS (EC2 · RDS · ECR · IAM/OIDC · EventBridge · Lambda) · Docker Compose
CI/CD	**GitHub Actions** (OIDC · matrix build) → Amazon ECR
Edge / Network	**Cloudflare** Tunnel · Workers · Universal SSL

11 · RETROSPECTIVE

회고 — 잘한 것과 의도적 선택의 트레이드오프

프로젝트를 마치며 두 갈래로 정리. **무엇을 잘했고, 어떤 부분이 트레이드오프를 따진 의도적 선택이었는지.** 각 선택의 한계와 다음 단계는 항목별 → **라인**에 함께 적음.

✅ 잘한 것

> 단순 CRUD를 넘어 **이벤트 유실(Outbox) · 최적화(VRPTW) · 동시성(비관적 락)** 등 운영 수준 문제를 **비즈니스 근거에서 출발해** 직접 설계. 스터디에서 학습한 락 트레이드오프를 실제 적용. "검증=동기, 통지=비동기"의 통신 의도 분리가 일관되게 관철됨.

🎯 의도적 선택

트레이드오프를 따져보고 현 규모에 적절하다 판단한 부분

배송 컨텍스트를 한 서비스로 통합 (delivery + manager + route)

초기 설계엔 `delivery / delivery_manager / delivery_route` 3개 서비스로 분할했으나, 실제 도메인 분석 결과 **3개 도메인이 동일 트랜잭션 경계에서 함께 변경**되는 강한 응집성을 보임 → 별도 서비스로 분리하면 분산 트랜잭션 비용만 증가한다고 판단해 **delivery-service 한 컨텍스트로 통합 + 내부 도메인으로 분리.**

→ "같은 트랜잭션에서 변하는 것은 같은 서비스로" — DDD의 애그리거트 경계 원칙을 실제 적용한 사례

Outbox 스케줄러가 단일 인스턴스 전제

분산 스케줄러·ShedLock·리더 선출 등 옵션을 검토했으나, **각 마이크로서비스가 단일 인스턴스로 배포되는 현 토폴로지**에선 스케줄러 중복 실행이 발생 X → 분산 락 도입은 오버엔지니어링.

→ 수평 확장 시점엔 ShedLock(DB 락) 또는 컨슈머 측 멱등 처리(이벤트 ID dedup) 추가 예정

full Saga 대신 Outbox 채택

다중 서비스 트랜잭션엔 Saga(보상 트랜잭션)도 후보였으나, 우리 케이스는 **단방향 통지·응답 불필요**라 full Saga의 복잡도가 과함.

→ 다중 단계 트랜잭션이 필요한 도메인에선 Saga가 정답 — 알고 있음

01 · OVERVIEW

LLM을 정량 평가하고, 안전하게 운영하고, 그 화면까지 만든 풀스택

모놀리식 → MSA → LLMOps로 이어진 설계 성장의 마지막 매듭. **RAG 답변 품질을 LLM-as-a-Judge로 정량 평가**하고, **Prometheus·Loki·Claude를 묶어 알림에 원인 분석과 복구 액션까지 자동 부착**하는 시스템을 설계·구현했습니다. AI를 외부 API로 덧붙이는 대신 **운영 모니터링 시스템의 일부로 통합한** 점이 차별점입니다.

> **"LLM을 검증 가능하게(Eval/Pairwise), 운영을 안전하게(HITL·다층 방어) 만든다."**
>
> 모의투자 플랫폼에서 도메인 사실성이 핵심인 RAG 챗봇 + LLM 기반 AIOps + 측정 가능한 관측 스택을 주도 설계.
>
> - 2026 · 부트캠프 5인
> - 담당: assistant · notification · monitoring + 프론트엔드(React·TS) 구축 주도
> - Java 17 · Spring AI 1.0 · pgvector · Kafka · GCP

2.38 → 4.04
faithfulness 향상
RAG 모델 교체 (LLM-as-a-Judge 점수 기반)

64.3% → 8.3%
환각률 −56pp
faithfulness < 3 비율

< 1분
MTTD
ServiceDown: scrape 15s × for 1m

74% ↓
캐시 히트 응답시간
3.68s → 0.95s (3.9배) · LLM 생성 호출 스킵

02 · SERVICE OVERVIEW

ants-camp — 개미배움캠프 · 가상 주식 투자 대회 플랫폼

*실제 주식 데이터를 기반으로, **가상 자산으로 투자 대회에 참여**하고 **수익률 경쟁으로 순위를 겨루는** 모의 투자 플랫폼.*

사용자는 한국투자증권 API로 가져온 **실시간 시세**를 보며 모의 매매를 진행하고, 매니저가 만든 대회 규칙(거래 시간·종목 제한·시드머니 등)에 따라 전략 경쟁을 합니다. 대회 종료 시점의 평가 자산 기반 **수익률로 순위·티어를 산정**하며, 단순 모의 투자를 넘어 **실시간 랭킹 · AI 챗봇 · LLM 기반 운영 알림 · MSA 확장 구조**까지 실제 서비스 수준의 경험을 목표로 설계했습니다.

서비스 6대 핵심 기능

01 · AUTH
회원 · 권한 관리
JWT 인증 + 사용자/매니저/관리자 RBAC. 매니저는 관리자가 등록한 계정으로만 활동.

02 · TRADING
가상 주식 매매
TradingView 실시간 조회 + 한국투자 API 기준가로 시장가 즉시 체결. 대회별 거래 시간·종목 제한 검증.

03 · COMPETITION
투자 대회 시스템
매니저 주도 대회 생성. 거래 규칙·참가 인원·시드머니 설정. 한 사용자가 여러 대회 동시 참가 가능.

04 · RANKING
실시간 랭킹 · 티어
주기적 수익률 계산 + Redis 캐싱. 종료 시 1·2·3등 발표 + TOP 10% 단위 티어 자동 산정.

05 · AI ASSISTANT ★ 담당
RAG 기반 챗봇 + 품질 평가
FAQ·약관·매매규칙을 벡터화해 의미 검색. LLM-as-a-Judge로 답변 품질 정량 측정.

06 · NOTIFICATION ★ 담당
통합 알림 + 모니터링
메트릭·로그 자동 수집 + LLM RCA → Slack HITL 액션. 중복 알림 SUPPRESSED 처리.

03 · DESIGN GOALS

설계 도전 과제와 핵심 기여

핵심 도전 과제

- RAG 답변의 **사실성(faithfulness)** 정량 측정 — 정량 **점수·승률**로 모델 결정
- LLM 평가의 2대 편향 차단 — **self-preference bias**(Judge가 자기 출력에 후한 경향) + **position bias**(Pairwise A/B 위치에 따른 편향)를 코드 레벨에서 모두 차단
- 단순 알림을 **"원인 + 권장 조치"**로 격상 — 운영자 컨텍스트 수집 단계 제거
- LLM 자동 실행의 위험 — 인프라/PII/위변조의 **다층 안전장치** 필요
- 토이 프로젝트의 한계 — **측정 가능한 관측 시스템** 자체를 차별점으로

핵심 기여 (개인)

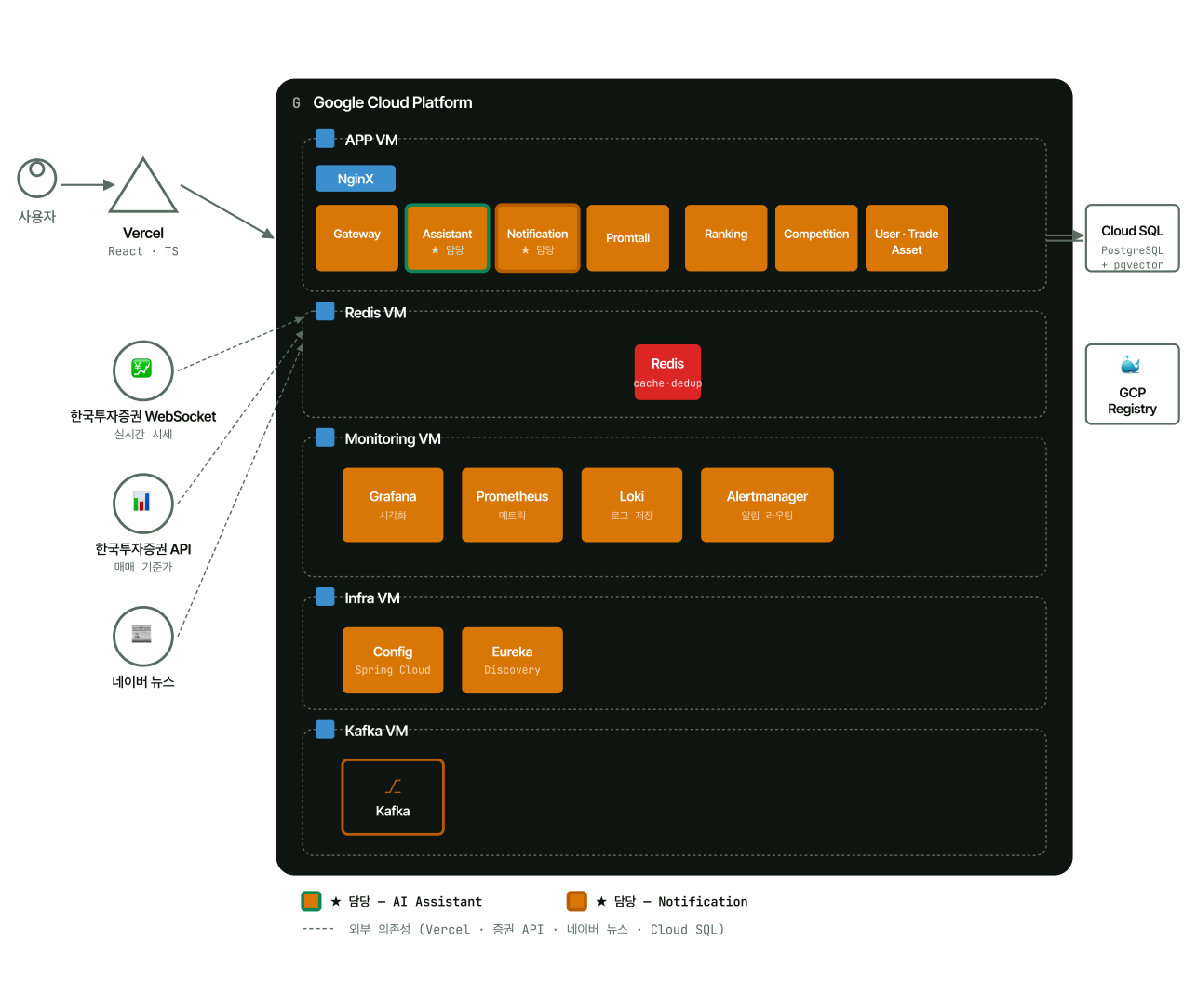
- RAG 파이프라인 + **LLM-as-a-Judge 3축 채점 + Pairwise** 시스템
- Self-preference + Position bias **코드 레벨 차단** + Judge 2종 교차
- Alertmanager → Redis dedup → Claude RCA → Slack **HITL** 파이프라인
- Defense-in-Depth: **인프라 보호 2중 차단 + HMAC-SHA256 + PII 마스킹**
- 관측 스택 주도 구축 — Prometheus/Loki/Promtail/Grafana/Alertmanager + alert 룰 **7종 `for:` 차등**
- RAG 응답 **pgvector 의미 캐시** — 히트 시 latency **74%↓** · LLM 호출 스킵

04 · ARCHITECTURE

TO-BE 아키텍처 — GCP 5-VM 분리 토폴로지

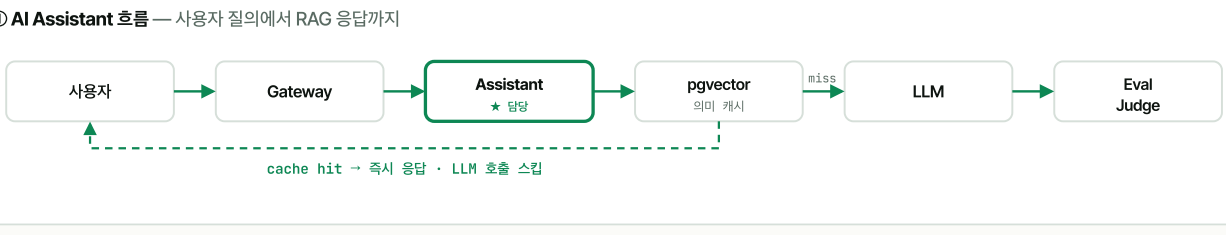
Google Cloud Platform 위에 **역할별 VM 5개 그룹**으로 분리 배포합니다. **APP VM**(서비스 컨테이너) · **Redis VM**(랭킹 캐시·dedup) · **Monitoring VM**(관측 스택 단독) · **Infra VM**(Config·Eureka) · **Kafka VM**(이벤트). 외부 의존성(**Vercel** 프론트 · **한국투자증권 API/WebSocket** · 네이버 뉴스 · **Cloud SQL** · **GCP Container Registry**)은 GCP 경계 바깥에서 통신. AI Assistant(녹색)와 Notification(주황) 두 흐름이 동일 인프라를 공유합니다.

> 위 구성은 **발표 시점의 아키텍처**입니다. 발표 종료 후 운영 비용을 줄이고자 백엔드·프론트엔드 전체를 **OCI 단일 인스턴스**(Docker Compose + Nginx)로 직접 마이그레이션했고, 현재 라이브 데모가 그 위에서 동작합니다.

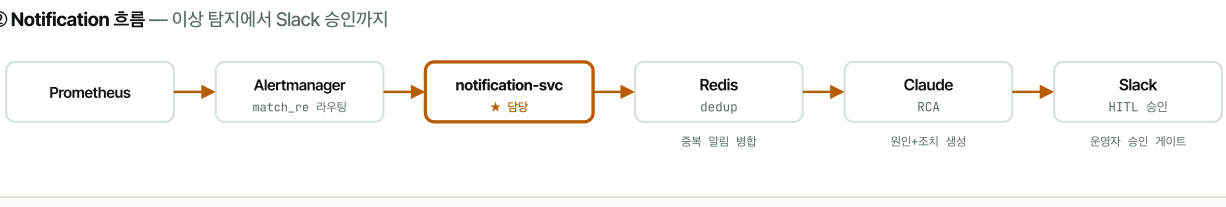


역할별 VM 토폴로지. 두 흐름의 단계별 상세는 아래 시퀀스 다이어그램 참고.

① AI Assistant 흐름 — 사용자 질의에서 RAG 응답까지



② Notification 흐름 — 이상 탐지에서 Slack 승인까지



설계 의사결정과 근거

결정	대안	선택 이유 — 정량 가정 · 확장 계획 포함
역할별 VM 5개 분리	단일 VM 통합 · K8s 클러스터	학습/포트폴리오 환경엔 K8s가 과함. 단일 VM은 SPOF·자원 경합 위험. **역할별 분리로 장애 격리와 자원 독립성 확보**. 운영 시 GKE로 단계적 이행 가능.
pgvector	Pinecone · Milvus	문서 수 수천 건 · QPS 한 자릿수 가정엔 cosine + IVFFlat로 p95 충분. **수십만 건 또는 멀티테넌시** 시점에 외부화 검토.
Spring AI + 모델명 라우팅	provider별 SDK 직접 호출	LLM(OpenAI/Claude/Gemini)을 Port 뒤로 추상화 → **모델 교체 시 application 무수정**. Pairwise 비교 실험 자유로움.
Claude Haiku (장애 분석)	Opus · Sonnet	알림은 빈도 높고 빠른 요약이 관건 → **Opus 대비 약 1/10 비용**, max_tokens 2000.
Human-in-the-Loop	LLM 전자동 실행	운영 컨테이너를 건드림 → **AI는 추천까지, 실행은 사람 클릭**. 자동화 효율과 인간 판단 안정성 사이의 균형.
Reconciler(@Scheduled)	메시지 큐 (DLQ)	인제스트 동시성 분당 수십 건 가정엔 60초 주기 보정으로 SLA 만족. **분당 수백 건 이상** 시 Kafka DLQ 전환.
단일 인스턴스 → 분산 락 생략	ShedLock · 리더 선출	각 서비스 단일 인스턴스 배포 토폴로지 → 스케줄러 중복 실행 자체 발생 X. **수평 확장 시** ShedLock + 컨슈머 멱등 처리.

05 · KEY CONTRIBUTION ①

🎯 CONTRIBUTION #1

LLM 답변 품질을 "감"이 아닌 "점수"로 검증 — 정량 평가 파이프라인

모델 교체 결정의 정량적 근거 확보. faithfulness 2.38 → 4.04, 환각률 64.3% → 8.3%, Pairwise 6승 0패

P · 대다수 RAG 프로젝트는 답변 품질을 "느낌"으로 판단하고 모델 교체를 "감"으로 결정 → 품질 회귀(regression) 위험을 막을 정량 체계 필요. 특히 LLM 평가의 가장 알려진 함정 **self-preference bias**(LLM이 자기 출력에 더 후한 경향, 연구로 보고됨)가 차단되지 않으면 평가 결과 자체가 신뢰 불가.

A · **검색 → 채점 → 비교** 3단계로 정량 평가 파이프라인 구성. self-preference bias는 **코드 레벨에서 차단** + Judge 2종 교차 검증으로 다중 방어.

> **검색** — pgvector 의미검색(topK×3 후 유사도 0.3·score-gap 0.8 컷오프 → 최종 topK=5)
>
> **채점** — Relevance / Faithfulness / ContextPrecision **3축** 점수화
>
> **비교** — Pairwise A/B 위치 교차(앞쪽 편향 회피)
>
> **bias 방어** — Judge==생성모델이면 채점 skip · Judge temperature 0 · Judge 2종(claude-sonnet-4-6 + gpt-4o) 교차 검증

R · 점수·승률 기반으로 **gpt-4o-mini → claude-haiku 교체. OpenAI 자사 gpt-4o조차 gpt-4o-mini를 낮게 평가** → 계열 편향 데이터로 반박.

CODE `EvalProcessor.java` · Judge 채점·bias skip `PairwiseProcessor.java` · A/B 위치 교차 `RetrievalReranker.java` · 검색 컷오프

★ 차별점 — 어떻게 차단했나 (3겹 방어)

① BIAS BLOCK
Self-preference 코드 레벨 차단
Judge 모델 == 응답 생성 모델이면 채점 자체를 **skip**. 자기 출력에 후한 경향을 회피.

② CROSS
Judge 2종 교차 검증
`claude-sonnet-4-6 + gpt-4o` 양쪽 모두 돌려 한 모델에 결정 의존 X. **OpenAI 자사 gpt-4o조차 gpt-4o-mini를 낮게 평가.**

③ PAIRWISE
A/B 위치 교차 비교
A/B 위치를 **교차해 양방향 비교** → 위치 편향(앞쪽 유리)까지 회피. 결과: 6승 0패.

▾ 핵심 코드 — Self-preference 차단 (Java) 클릭해 펼치기

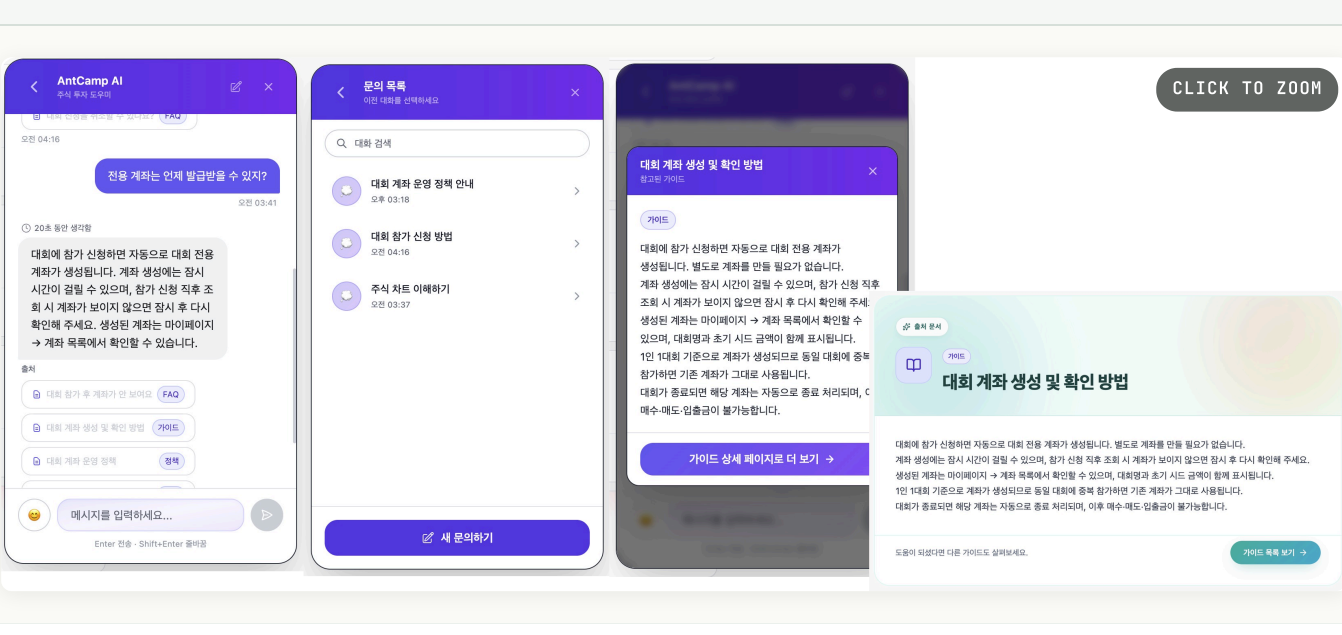
```java
// EvalProcessor.judgeAndSave()
if (judgeModel.equalsIgnoreCase(ragQuery.getLlmModel())) {
    log.warn("Self-preference bias 방지: skip. model={}", judgeModel);
    return;
}
```

결과 — 데이터 기반 의사결정

지표	GPT-4O-MINI (N=28)	CLAUDE-HAIKU (N=12)
faithfulness	2.38	4.04
relevance	2.38	4.00
contextPrecision	2.32	3.92
환각률 (faith < 3)	64.3 %	8.3 %
Pairwise (A/B 교차)	0승	6승 0패 (TIE 0)

표본 40건은 통계적 검정력 충분치 않으나 ① 두 Judge 동일 방향 ② Pairwise 동일 방향 → 방향성 견고. 정밀화 시 골든셋 200건+ 확대 및 사람 평가자 inter-rater agreement 측정으로 Judge 신뢰도 자체 검증 계획.

📸 평가 대상이 된 실제 RAG 챗봇 UI RAG · 출처 표시



위 정량 평가 시스템(**LLM-as-a-Judge · Pairwise)의 평가 대상**이 된 사용자 챗봇 화면. 응답에 더해 **참조 문서의 출처를 함께 표시**해 사용자가 근거를 직접 확인할 수 있도록 설계했습니다.

▾ 이 화면에서 보이는 것 클릭해 펼치기

- ✓ **질문 → 응답 → 출처 가이드** 3단계 표시 — "전용 계좌는 언제 발급받을 수 있죠?" 질문에 RAG로 검색한 가이드 문서 기반 답변 생성
- ✓ **출처 칩 클릭** → 우측 패널에서 참조한 가이드 문서의 주요 내용 미리보기
- ✓ **"가이드 상세 페이지로 더 보기"** 버튼 → 전체 문서 페이지로 이동, 답변의 근거를 원문까지 추적 가능
- ✓ 세션 기반 대화 이력 관리 — 좌측 "문의 목록"에서 이전 대화 재조회 (세션 제목은 마지막 메시지로 자동 설정)

06 · KEY CONTRIBUTION ②

📊 CONTRIBUTION #2

측정 불가능한 운영을 측정 가능한 운영으로 — 관측 스택 주도 구축

MTTD < 1분 · alert 룰 7종 차등 설계 · 반복 알림 LLM 비용 0

P · 토이/부트캠프 프로젝트는 **측정 시스템 부재로 정량 수치를 못 남기는** 경우가 많음. *"측정 가능한 시스템을 처음부터 만들었다"*는 것 자체가 차별점이며, alert 룰 설계 역시 노이즈와 심각도를 함께 고려해야 함.

A · 관측 스택을 주도 구축하고 **alert 룰 7종을 `for:` 지속조건으로 차등 설계** — "터지면 큰가(심각도) × 자주 출렁이나(노이즈)"를 함께 반영.

> **수집** — Prometheus(15s scrape · 10 job) + Promtail(`docker_sd_configs` 컨테이너 자동 발견)
>
> **저장** — Loki(로그) · Prometheus TSDB(메트릭)
>
> **시각화** — Grafana provisioning IaC(대시보드 코드 관리)
>
> **알림** — Alertmanager + 룰 7종 차등 설계(아래 표)

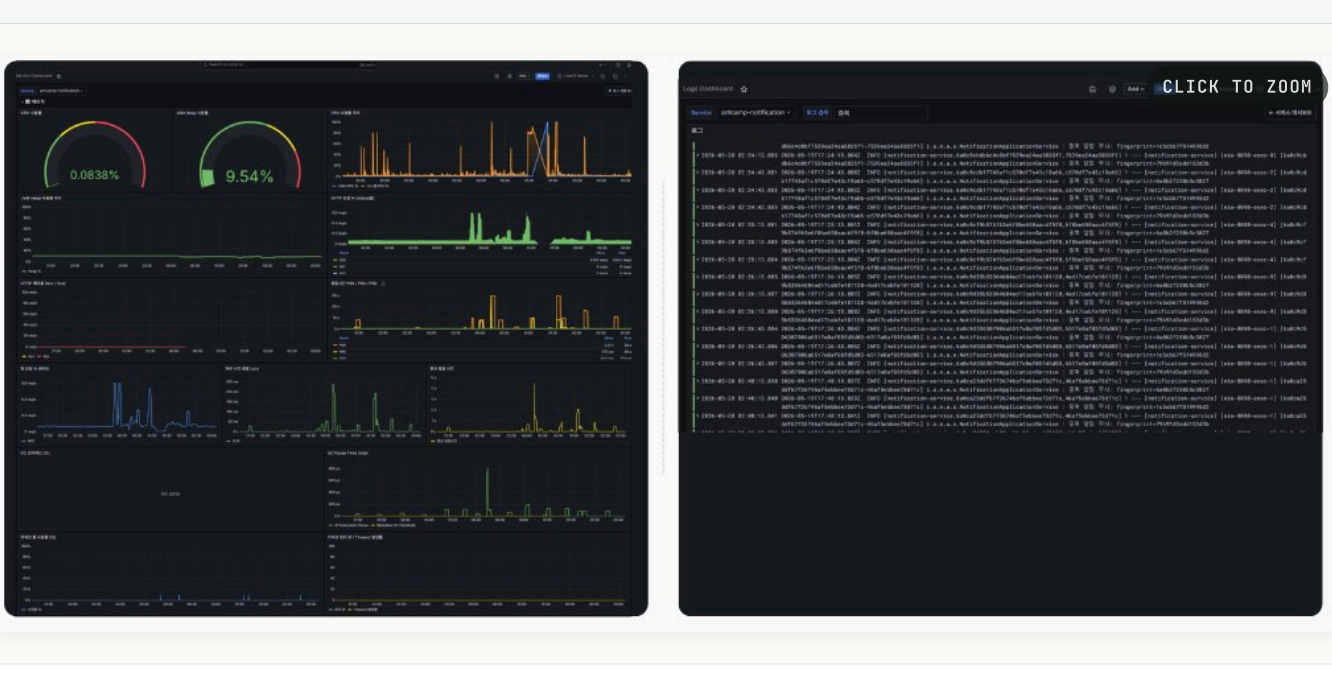
R · MTTD < 1분, 반복 알림 LLM 비용 0(dedup). SPOF 해소 fallback까지 완비.

CODE `alert-rules.yaml` · 룰 7종 for: 차등 `alertmanager.yaml` · match_re fallback `promtail.yaml` · docker_sd `monitoring/` · 스택 전체

룰	임계 / FOR	심각도	근거
ServiceDown	`up == 0 / 1m`	critical	scrape 15s × 4회 연속 실패해야 발동 → 순간 끊김과 구분
ConnPoolPending	`> 3 / 1m`	critical	대기 자체가 고갈 신호 → 가장 짧게
HighCPU	`0.8 / 2m`	critical	throttling 시작 구간, 100% 직전 여유
HighMemory · ConnPoolNear · HttpError	`/ 2m`	warning	스파이크와 추세 구분 가능한 지점
HighGcOverhead	`0.1 / 5m`	warning	단기 변동 잦아 가장 길게

★ **SPOF 해소** — notification-service 자기 장애 시 Alertmanager `match_re` (라벨 정규식 매칭으로 알림 라우트 분기) fallback으로 Slack 직통(LLM 분석 없는 단순 텍스트로 알림 보존).

📸 실제 운영 중인 Grafana 통합 대시보드 관측 스택 · 실측



"측정 가능한 시스템을 만들었다"의 직접적 증거. 좌측은 **서비스 대시보드**(Prometheus 메트릭), 우측은 **로그 대시보드**(Loki 집계)로 **메트릭과 로그를 Grafana 단일 UI에서 통합**해 운영자가 화면을 오가는 수고를 덜었습니다.

▾ 이 화면에서 보이는 것 클릭해 펼치기

- ✓ **핵심 게이지**(좌상단): CPU 사용률 0.0838% · JVM Heap 9.54% — 실시간 상태를 게이지로 즉시 가독
- ✓ **시계열 패널 다수**: CPU/메모리/HTTP 응답 시간/요청 수/스레드/GC/JVM CPU 사용률 등 **서비스 헬스의 다축 추적**
- ✓ **로그 통합 뷰**(우측): notification-service의 INFO/WARN 로그 실시간 스트림. **로그 라벨링**으로 컨테이너별·서비스별 필터링 가능

왜 PGAL Stack인가 — 모니터링 도구 선택 근거

MSA 환경의 주요 비기능 요구는 **장애 추적·실시간 메트릭·중앙 집중 로그·자동 알림**입니다. ELK · CloudWatch · Datadog 등 후보를 **운영 비용 · 의존성 · 우리 규모 적합성** 세 축으로 비교한 결과, **Prometheus + Grafana + Alertmanager + Loki + Promtail**(이하 PGAL)이 적절하다고 판단했습니다.

▾ 도구 선택 근거 상세 — 왜 PGAL Stack인가 클릭해 펼치기

ELK Stack
Elasticsearch + Logstash + Kibana
✓ 강력한 Full-text 검색 · 대규모 로그 분석 · Kibana 시각화 우수
✗ Elasticsearch 메모리 사용량이 매우 큼 · Logstash 리소스 무거움 · 운영 복잡도 높음
판단: 우리는 Kubernetes 규모의 초대형 시스템이 아니고, 로그 분석보다 **장애 추적이 더 큰 목적**. Docker 기반 경량 MSA엔 과한 구성.
REJECTED (over-spec)

CloudWatch
AWS managed
✓ AWS 서비스와 높은 통합성 · 별도 서버 운영 불필요 · 자동 스케일링
✗ AWS 의존성 증가 · 로그 저장 비용 누적 · 멀티 클라우드 대응 어려움
판단: 본 프로젝트는 **GCP + 로컬 Docker 기반**으로 운영 — 특정 클라우드 종속 회피.
REJECTED (vendor lock-in)

Datadog
SaaS APM
✓ SaaS 구축 쉬움 · UI/UX 우수 · 로그+메트릭+트레이싱 통합 APM
✗ 비용이 높음 · 데이터량 증가 시 과금 부담 · SaaS 의존성
판단: 학습/포트폴리오 환경에서는 **운영 비용 최소화가 중요**해 부적합.
REJECTED (cost-prohibitive)

PGAL Stack
Prometheus + Grafana + Alertmanager + Loki + Promtail
✓ MSA 사실상 표준(Prometheus) · Spring Boot Actuator 연동 용이 · ELK 대비 경량(Loki) · Grafana 단일 대시보드로 메트릭+로그 통합 · Docker SD로 컨테이너 자동 발견 · 전부 오픈소스로 비용 0
판단: 우리 규모(Docker 기반 MSA · 5인 팀 · 학습/데모 환경)에 정확히 맞는 균형. **운영 가시성을 확보하면서도 운영 부담을 최소화.**
CHOSEN ★

▾ 5개 도구 각각의 역할 (Prometheus · Grafana · Alertmanager · Loki · Promtail) 클릭해 펼치기

📊 **Prometheus** 시계열 메트릭 scrape 15s
📈 **Grafana** 통합 대시보드 IaC provisioning
🚨 **Alertmanager** 조건 기반 알림 Slack 라우팅
📜 **Loki** 경량 로그 저장 라벨 인덱싱
🚚 **Promtail** 컨테이너 로그 Docker SD

도구	선택 이유 (한 줄)	활용 목적
Prometheus	MSA 사실상 표준 · Spring Boot Actuator 연동 용이 · 시계열 저장 최적화 · PromQL	CPU/Memory/API 응답시간/에러율/요청수 추적
Grafana	다양한 데이터 소스 + Loki/Prometheus와 높은 호환 + 대시보드 자유도	서비스별 메트릭 시각화 · 운영 대시보드 · 장애 실시간 확인
Alertmanager	Prometheus 자연스러운 통합 · 조건 기반 · Slack 연동 용이	임계치 초과 → Slack 알림 · API Error Rate 증가 감지
Loki	Elasticsearch 없이 로그 저장 · 라벨 인덱싱으로 비용 절감 · Docker 궁합	서비스별 중앙 로그 · 장애 로그 분석 · Trace ID 요청 추적
Promtail	Docker 로그 수집 단순 · Loki 공식 연동 · 설정 단순	컨테이너 로그 수집 · 서비스 라벨링 · Loki로 전달

07 · KEY CONTRIBUTION ③

🛎 CONTRIBUTION #3

단순 알림을 넘어 원인 분석을 전달 — LLM 기반 AIOps (Human-in-the-Loop)

MTTR 단축 + AI 자동화의 안전성을 사람 클릭으로 통제. 운영자의 컨텍스트 수집 단계 제거

P · Prometheus → Slack 단순 알림은 "임계값 초과" 사실만 전달 → 운영자가 Grafana·터미널을 왕복하며 원인 추적에 시간 소요(MTTR의 대부분). 동시에 LLM 자동 실행은 운영 컨테이너를 건드리므로 위험.

A · Alertmanager webhook부터 Slack 메시지까지의 파이프라인을 **dedup → 컨텍스트 수집 → LLM 분석 → HITL 액션** 4단계로 구성. **AI는 추천까지, 실행은 사람 클릭**이라는 안전선 확보.

> **dedup** — Redis fingerprint(`SET NX` , TTL 1h)로 중복 알림 차단
>
> **컨텍스트 수집** — Prometheus(CPU/Heap/에러수/응답시간) + Loki(최근 10분 에러 로그) 자동 fetch
>
> **LLM 분석** — Claude(haiku)가 메트릭·로그 기반 RCA(원인 분석 + 권장 조치) 생성
>
> **HITL 액션** — Slack Block Kit으로 분석 + **복구 버튼 4종**(restart / rollback / cache-clear / false-alarm) 동시 노출

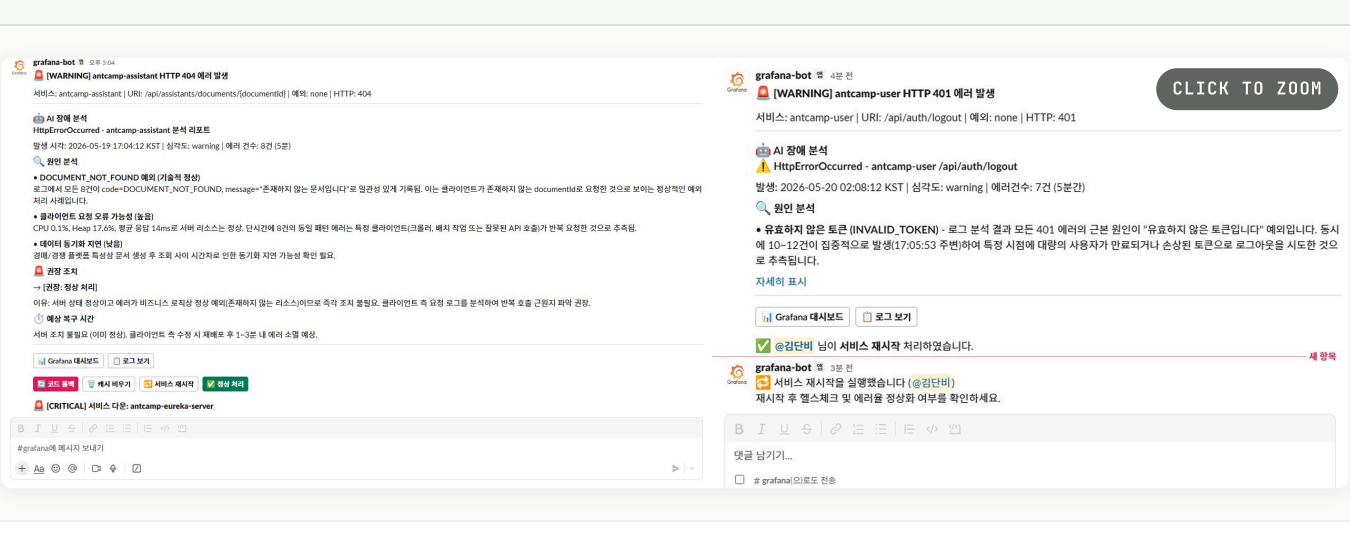
R · 알림 수신 후 수 초 내 "근거 메트릭/로그 + 권장 조치" 자동 게시. **AI를 운영 모니터링 시스템의 일부로 통합.**

▾ 핵심 코드 — @Async 비동기 ACK (Java) 클릭해 펼치기

```java
// Slack 3초 응답 제약 → @Async ACK 후 메시지 갱신
@Async("slackActionExecutor")
public void executeAndNotifyAsync(SlackActionCommand command) {
    trySlack("처리 중", () -> alertPort.markAsProcessing(...));
    ActionResult result = executeAction(...);
    trySlack("완료/실패 갱신", () -> alertPort.markAsHandled(...));
}
```

CODE `NotificationApplicationService.java` · dedup→RCA→HITL `SlackBlockBuilder.java` · 복구 버튼 4종 `ClaudeApiClient.java` · LLM RCA `RedisDeduplicationAdapter.java` · SET NX dedup

📸 실제 Slack 알림 + Human-in-the-Loop 액션 화면 HITL · 실제 운영



실제 Slack 채널에 자동 게시된 **LLM 기반 장애 분석 메시지**와 복구 버튼 클릭 후 스레드 갱신까지의 동작 흐름.

▾ 이 화면에서 보이는 것 클릭해 펼치기

- ✓ **AI 장애 분석 리포트** — antcamp-user HTTP 400 에러 발생 시점, 심각도(warning), 에러율(7건/5분)을 자동 포착
- ✓ **원인 분석** — 1순위 클라이언트 입력값 검증, 2순위 일시적 클라이언트 트래픽, 3순위 서버 상태 등 **우선순위가 매겨진 분석** 제공
- ✓ **복구 버튼 4종**(빨강 박스 영역): 코드 롤백 · 캐시 비우기 · 서비스 재시작 · 정상 처리(false alarm) — 운영자가 상황에 맞게 선택
- ✓ **스레드 자동 갱신**(우측) — 버튼 클릭 시 `@Async` ACK 후 "처리중 → 완료/실패"로 메시지 갱신. Slack 3초 응답 제약 해소
- ✓ **Grafana 대시보드 / 로그 보기** 바로가기 버튼으로 운영자의 컨텍스트 전환 비용 최소화

08 · KEY CONTRIBUTION ④

🛡 CONTRIBUTION #4

LLM이 운영 시스템을 만지는 위험을 다층으로 차단 — Defense-in-depth

장애 시나리오 3종에서 fallback 동작 검증 완료

P · LLM 제안 시스템은 ① 인프라 서비스 오작동(LLM이 gateway/kafka에 destructive action 권유), ② 외부 위변조 요청(공개 endpoint 노출), ③ LLM 입력으로의 PII/시크릿 유출 등 다층 위험을 동시에 가짐.

A · 세 위험에 각각 독립적 방어선을 둠 (아래 3겹).

R · 세 시나리오 모두 fallback 동작 검증 완료 — ① notification-service 자기 장애 시 Alertmanager `match_re` fallback으로 Slack 직통, ② LLM API 장애 시 단순 텍스트 fallback, ③ 비동기 ingest 실패 시 Reconciler 자가복구.

CODE `SlackSignatureVerificationFilter.java` · HMAC·replay·timing `PromptUtil.java` · PII 마스킹 `NotificationProperties.java` · 인프라 화이트리스트

① INFRA PROTECTION
인프라 서비스 2중 차단
gateway·kafka 등 `infrastructureJobs` 는 SlackBlockBuilder에서 **버튼 자체 미노출** + executeAction 진입 시 **화이트리스트 차단.** API/payload 조작 우회까지 방어.
→ 두 겹 중 한 겹이 뚫려도 다음 겹이 막음

② SLACK FORGERY
위변조 · Replay · Timing 공격 방어
공개 endpoint에 전용 필터: **HMAC-SHA256 서명 검증** + timestamp 5분 초과 거부(replay) + `MessageDigest.isEqual` constant-time 비교(timing attack). body는 `CachedBodyRequest` 래핑.
→ 외부 위변조 요청 원천 차단

③ PII MASKING
LLM 입력 민감정보 마스킹
token · password · secret · bearer → `[REDACTED]` , 이메일 → `[EMAIL]` , 로그 4000자 제한. 외부 LLM 호출 직전 최종 검증.
→ 시크릿/PII 외부 API 유출 방지

09 · FRONTEND

한 기능을, 서버부터 화면까지 끝까지

사용자 화면과 관리자 콘솔 전체를 React·TypeScript·Tailwind로 구축했습니다. 백엔드에서 만든 RAG 평가·AIOps 기능을 운영자가 실제로 쓰는 화면까지 이어 만든 것이, 이 프로젝트에서 풀스택으로 일한 방식입니다.

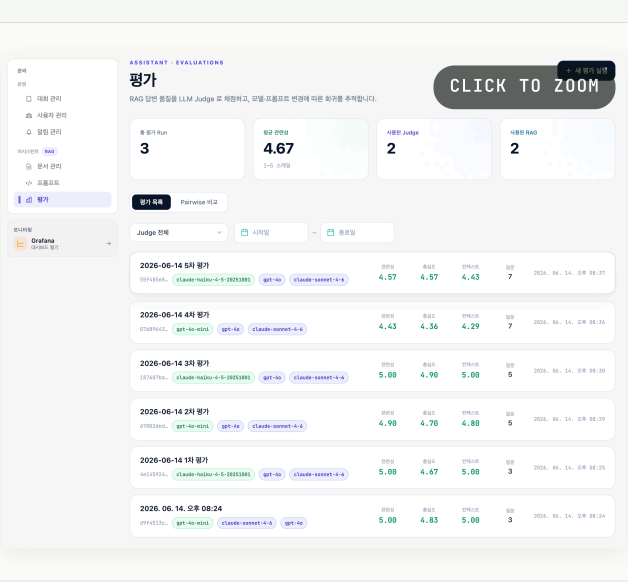
사용자 화면

- 실시간 시세 **티커·캔들 차트** (Recharts) · 종목 검색·상세
- **매매** — 호가창 + 주문 패널, WebSocket(STOMP) 시세 구독
- **투자 대회** — 목록·상세·실시간 순위 대시보드
- 포트폴리오·마이페이지 — 보유 종목·평가 손익
- **AI 챗봇** — RAG 답변 마크다운 렌더링

관리자 콘솔

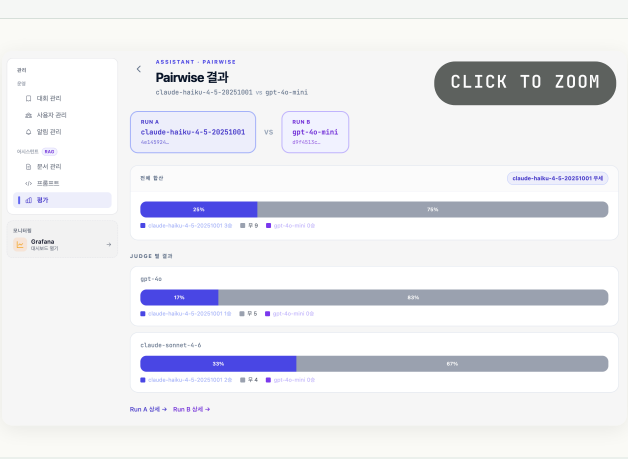
- 회원·대회 관리 — 검색·필터, 대회 규칙 설정
- **RAG 운영 콘솔** — 문서 등록·평가 실행
- **Pairwise A/B 비교 결과 시각화** · 프롬프트 버전 관리
- 알림 관리 — AIOps 알림 목록·상세

📸 RAG 운영 콘솔 — 평가·문서 관리 Admin · React



문서 등록·평가 실행·프롬프트 버전을 한 화면에서 다루는 **운영자용 콘솔**. 백엔드 평가 파이프라인의 입력과 결과를 화면으로 연결했습니다.

📸 Pairwise A/B 비교 결과 평가 시각화



두 모델·프롬프트의 응답을 **승/패로 시각화**해, 운영자가 평가 결과를 보고 교체를 결정하도록 한 화면. "감이 아닌 점수로" 모델을 고르는 과정을 그대로 담았습니다.

> **백엔드와 프론트엔드를 잇는 한 가지 — RAG 운영 콘솔**
>
> LLM-as-a-Judge 평가 파이프라인(백엔드)부터 그 결과로 모델 교체를 결정하는 콘솔(프론트)까지 직접 구현했습니다. 덕분에 **화면이 필요로 하는 형태로 API를 설계**할 수 있었고, 평가 → 비교 → 적용으로 이어지는 운영 흐름이 화면 안에서 끊기지 않습니다.

상태 관리 — Redux vs Zustand. Redux의 강점(예측 가능한 단방향 흐름, 미들웨어·DevTools 생태계, 대규모 팀 협업 규약)과 비용(보일러플레이트, 러닝커브)을 이 프로젝트 조건에 비춰봤습니다. 전역 상태가 **인증·시세·테마**로 한정적이고 5인 단기 프로젝트라, Redux의 구조적 이점을 살릴 규모가 아니라고 판단해 **가벼운 Zustand**를 택했습니다. "왜 이 기술인가"를 설명할 수 있을 때 고른 선택입니다.

CODE `ants-camp-frontend` · React · TS · Tailwind · Zustand ▶ 라이브 데모 · 게스트 로그인

10 · TROUBLESHOOTING

트러블슈팅 — 원인을 윗단까지 파고든 사례

"외부 호출 때문에 트랜잭션이 길어졌다"는 표면적 설명에서 멈추지 않고, **커넥션 풀 → 톰캣 워커 스레드 → 세션 한계**까지 원인을 윗단으로 추적한 사례들.

#01 · LLM 호출 중 DB 커넥션 점유 → 풀 고갈 위험 트랜잭션 경계 분리

현상 · 가설	원인 분석 (WHY → WHY)	해결 & 검증
→ RAG 답변 생성은 평균 수 초 소요 → 유저메시지·봇메시지 저장을 단일 `@Transactional` 로 묶으면 **LLM 응답 대기 내내 HikariCP 커넥션 점유** → 동시 요청 시 풀 고갈 → 톰캣 워커 스레드까지 블로킹 → 전체 타임아웃	→ LLM 호출이 트랜잭션 안 → 커넥션 점유 → 커넥션 풀 고갈 → 톰캣 스레드도 트랜잭션 잡은 채 블로킹 → 스레드 풀 한계 도달 → 새 요청 거절 → 풀 확장은 근본 해결 X — 요청량 늘면 다시 고갈	→ **3구간 분리**: 유저메시지 저장(T1) → LLM 호출(**트랜잭션 없음**) → 봇메시지 + COMPLETED(T2) → omin 프로젝트 트랜잭션 경계 분리 교훈을 LLM 호출에 그대로 재사용 → *측정 후속 계획: Micrometer `hikari_connections_active` + `llm_call_duration` 게이지로 점유 시간 정량화 예정*

CODE `RagApplicationService.java` · T1→LLM(무TX)→T2

#02 · 비동기 ingest 부분 실패 → DB · 벡터 불일치(orphan) 멱등성 · 자가복구

현상 · 가설	원인 분석	해결 & 검증
→ 문서 인제스트는 **청킹 → 임베딩 → 저장** 3단계 → 임베딩 단계 실패 시 청크는 DB에 남고 벡터는 비어 orphan 발생 → 재인제스트 시 단순 INSERT는 중복 벡터 생성	→ 다단계 비동기 처리 + 부분 실패 시점이 불특정 → 재시도 시 멱등성 미보장 → 중복 가능 → 실패 작업이 PROCESSING 상태로 영구 잔존 위험	→ 재인제스트 전 **기존 벡터 선삭제**(멱등성) → `DocumentReconciler` 60s 주기로 10분 PROCESSING 감지 → 최대 3회 재시도 · PERMANENT 격리

CODE `DocumentReconciler.java` · 60s 보정·3회·PERMANENT

#03 · 반복·유사 질문에도 매번 풀 RAG → 불필요한 LLM 지연·비용 pgvector 의미 캐시

현상 · 가설	접근 — 의미 캐시	해결 & 검증
→ RAG 답변은 LLM 생성에 평균 수 초 소요 → FAQ·약관·매매규칙처럼 **동일·유사 질문이 반복**되는 도메인 → 매번 풀 RAG = 지연·비용 낭비	→ 첫 턴 질문을 임베딩해 **pgvector 의미 캐시** (`p_response_cache`) 조회 → `1 - cosine ≥ threshold` & 미만료면 저장 답변 즉시 반환 → **LLM·벡터검색 생략** → 미스 시 풀 RAG 후 적재(TTL config) · 첫 턴(히스토리 빈 상태)에서만 → 단발성 트래픽 최적	→ 캐시 히트 **0.95s** — 미스(긴 답변 3.68s) 대비 **74%↓ (3.9배)** → LLM 생성(2.41s) 자체를 제거 → 워크로드별 wall-time·LLM `latencyMs` 분리 측정(아래 표)

CODE `ResponseCacheAdapter.java` · 의미 캐시 조회/적재 `RagApplicationService.java` · 첫 턴 캐시 흐름

구분	클라이언트 WALL-TIME (중앙값)	서버 LLM LATENCYMS
미스 — 긴 답변(100–185자)	3.68s	2.41s
미스 — 짧은 거절(82자)	2.82s	1.34s
캐시 히트	0.95s	LLM 호출 스킵

동시성 · 같은 응답 경로에서 동일 세션 동시 전송 시 seq 채번 race는 `findMaxSeqForUpdate` + `@Lock(PESSIMISTIC_WRITE)` 로 차단 — 락 선택 트레이드오프는 배송 담당자 seq 채번에서 상세히 다룬 동일 판단의 재적용.

11 · STACK

기술 스택

영역	스택
Backend	Java 17 · Spring Boot 3.5 · Spring Cloud (Eureka · Config · Gateway) · **Spring AI 1.0**
Frontend	**React · TypeScript · Tailwind CSS** · Zustand · Recharts
AI / LLM	**LLM-as-a-Judge · Pairwise · pgvector** · Claude (Anthropic) · OpenAI · Gemini (fallback)
Data	PostgreSQL + pgvector · Redis · Kafka
Observability	Prometheus · Grafana · Loki · Promtail · Alertmanager · Zipkin
Infra	Docker · Terraform · GCP · GitHub Actions

12 · RETROSPECTIVE

회고 — 의도적 선택과 진짜 아쉬운 점을 구분하다

프로젝트를 마치며 세 갈래로 정리. **무엇을 잘했고, 어떤 부분이 의도적 선택이었으며, 어디가 진짜 아쉬웠는지.**

✅ 잘한 것

> LLM을 붙이고 끝내는 대신 **검증 가능한 시스템**(Eval/Pairwise · bias 차단)으로, 운영을 **안전하게**(Human-in-the-Loop · 다층 방어 · 자가복구) 만든 것. 그리고 토이 프로젝트의 한계인 "측정 수치 부재"를 **관측 스택 주도 구축**으로 정면 돌파.

🎯 의도적 선택

트레이드오프를 따져보고 현 규모에 적절하다 판단한 부분

복구 액션이 단일 호스트 Docker socket 1:1
SSM · K8s rolling restart로 외부화하는 옵션도 검토했으나, **단일 호스트 데모 환경**에선 직접 호출이 더 단순·명료. 운영 복잡도 증가의 비용이 이득을 넘어선다고 판단.
→ 다중 인스턴스 시점엔 SSM/K8s API로 외부화 예정

Loki `replication_factor=1` · retention 미설정
단일 VM 토폴로지에선 HA가 의미 없음 → 단순 구성 선택. 데이터 영속성도 데모 범위 안에서 충분.
→ 운영 환경 전제로는 다중 노드 + 보관 정책 도입

인증을 게이트웨이 헤더에 위임
모든 서비스에 JWT 검증을 중복 구현 vs 게이트웨이 단일 검증의 단순함을 비교한 결과 **부트캠프 규모엔 후자가 적절**.
→ 운영 환경엔 서비스 레벨 JWT 한 겹 추가가 정석 — 알고 있고 다음에는 그렇게 설계

분산 락 생략
각 서비스가 단일 인스턴스 배포 토폴로지에선 스케줄러 중복 실행 자체가 발생 X → ShedLock 도입은 오버엔지니어링.
→ 수평 확장 시점에 ShedLock + 컨슈머 멱등 처리 추가

🛠 진짜 아쉬운 것

더 챙겼어야 하는 부분

의미 캐시 임계값이 엄격 — 유효 hit율 한계
측정해 보니 코사인 ≈0.95 임계값이 사실상 **동일 문구만 통과**시켜, 같은 의도라도 표현이 다른 질문은 전부 미스(paraphrase hit율 0%). 캐시 히트도 ~0.95s 소요(질문 임베딩 왕복 1회). 측정 자체도 단일 클라이언트·소표본 기준이라 동시성/부하 하 수치는 아님.
→ 임계값 0.90~0.92 완화(미묘하게 다른 질문에 오반환 위험과 트레이드오프) · 정규화 질문 exact-match 단축 경로 · 임베딩 캐싱 검토

01 · OVERVIEW

AI가 일하는 방식을 설계하는 것 — 하네스 엔지니어링

개발을 AI에게 일임하는 것이 아니라, **AI가 이 코드베이스 규칙을 벗어나지 못하도록 통제·감독하는 체계**를 설계합니다. AI의 결과물은 빠르고 그럴듯하지만 **맥락을 모른 채 자주 틀리기에,** `antcamp` (가상 주식 거래 MSA · 도메인 서비스 7개)에 **규칙 문서(SSOT) · 자동 검토/테스트 스킬 · 가드레일**을 둘러 **AI의 출력은 사람의 검증을 통과해야만 코드베이스에 반영되도록** 했습니다. 코드의 **최종 판단과 책임은 항상 사람**이 집니다.

harness engineering — AI를 통제·감독하는 체계를 엔지니어링

반복 타이핑만 AI로 처리하되, **무엇을·언제 참조할지, 무엇을 실행하면 안 되는지, 무엇을 반드시 검증할지**를 사람이 못박습니다. AI가 단독으로 코드를 확정하는 일은 없습니다 — 모든 출력은 규칙·자동 검토·빌드 검증을 거쳐 **사람이 최종 확인**합니다. 도구(모델·CLI)는 빠르게 바뀌지만, 규칙 문서·검토 스킬·가드레일이라는 **관리 체계**는 프로젝트를 넘어 남습니다.

- SSOT 규칙 문서 3종
- 자동 검토·검증 A · B · C
- 합격 판정은 사람이

02 · DEV CYCLE

개발 사이클 단계별로 도구를 나눠 쓴다

분석·구현·리뷰는 요구하는 판단이 다릅니다. 단계마다 도구를 분리하되, **설계·결정과 핵심 분기는 사람이 직접 운전**하고 AI의 출력은 그 자리에서 검증합니다.

분석 · 설계

Claude Code

루트 `CLAUDE.md` 의 헥사고날 4레이어·네이밍 규약을 기준으로 호출 경로와 설계 대안을 교차 검증. 포트를 안쪽(application/domain)에 먼저 두고 어댑터를 `infrastructure` 에 두는 흐름을 그대로 따름.

구현 · 테스트

Claude Code · 서브에이전트

보일러플레이트·반복 작성에만 서브에이전트를 쓰고, **도메인 로직과 메인 흐름은 직접 작성·통제.** AI가 낸 초안은 경계 조건·검증 기준을 다시 설계해 **그대로 받지 않고 반드시 손본다.**

리뷰

/revref · CodeRabbit

커밋 전 직접 만든 `/revref` 스킬이 컨벤션 위반을 점검(신규 파일·50줄+ 수정 시 자동), PR 단계는 **CodeRabbit**이 동일 원칙으로 한 번 더. **반복 지적은 규칙 문서로 흡수.**

03 · RULES · SSOT

규칙을 단일 출처(SSOT) 문서로 주입한다

AI가 일관되게 일하려면 "무엇이 규칙인가"가 **한 곳에** 있어야 합니다. 세 문서가 서로를 참조하며 **구조 · 규칙 · 절차**를 나눠 가지고, CI 리뷰 봇(`.coderabbit.yaml`)과 **동일한 기준**을 공유합니다.

구조 · 컨벤션

CLAUDE.md

헥사고날 4레이어, **패키지·네이밍 표**, 게이트웨이 인증 흐름(`X-User-Id` 신뢰 경계), Config Server 외부화, `common` 모듈 규약.

코딩 규칙

coding-conventions.md

트랜잭션 범위·JPA 캡슐화·DDD/레이어 경계·MSA 경계· `record` VO·동시성/분산락·로깅 보안·common 재사용. 위반은 **심각도**로 분류.

절차

automation-workflow.md

코드 작업 후 **자동 검토(A) · 빌드/테스트/API(B) · Slack 보고(C)** 워크플로우의 트리거와 흐름을 정의. "언제 무엇을 자동으로 할지"를 명문화.

심각도 체계 — 리뷰가 무엇을 막고 무엇을 권하는가

🔴 `Request Changes` 반드시 수정

- **아키텍처 경계 위반** — DDD/레이어/트랜잭션/이벤트 경계
- `@Transactional` 안에서 외부 API 호출(커넥션 점유)
- JPA 엔티티 `@Setter` / `@Data` — 상태 변경은 의미 있는 메서드로
- 1 트랜잭션 1 Aggregate · 분산 트랜잭션(2PC) 금지 → Kafka 최종 일관성
- 동시성 제어 누락 · 민감정보 평문 로그

🟡 `Comment` 개선 권장

- 성능 · 코드 품질 · 리팩토링
- VO/DTO/Event는 `record` · Strong Type VO(primitive obsession 회피)
- N+1 회피 — 연관관계 `LAZY` 기본, 필요 시 fetch join

코드베이스 특수 예외는 규칙으로 흡수한다

일반론이 이 코드베이스엔 안 맞는 경우, **왜 예외인지를 규칙 문서에 직접 박아** AI 리뷰가 같은 지적을 반복하지 않게 합니다. 예: "모든 `record` 의 compact constructor에 검증을 넣으라"는 일반 규칙에 대해 —

conventions §4 — 명문화된 예외

- 서비스 간 **내부 이벤트 메시지 `record` 는 발행 측에서 검증되었다고 가정**하므로 compact constructor 검증의 예외로 둔다.
- → AI/리뷰 봇이 "검증 누락"으로 잘못 지적하지 않고, **의도한 설계**임을 그대로 확인할 수 있다.

04 · WORKFLOWS

작업 단계에 맞춰 자동으로 도는 워크플로우 A · B · C

규칙은 "지켜야 한다"로 끝나지 않게, **작업이 끝나면 자동으로 검토·검증·보고**가 돌도록 슬래시 스킬과 스크립트로 묶었습니다.

워크플로우	트리거	수단
A · 코드 작업 후 자동 검토	새 파일/메서드 추가, 50줄+ 수정, `common/` 변경	`/revref` 스킬
B · 빌드→테스트→API 실행	런타임 동작에 영향 주는 변경 완료	`/apitest` 스킬
C · 완료 후 Slack 보고	A/B 또는 주요 작업 완료	`notify-slack.sh`

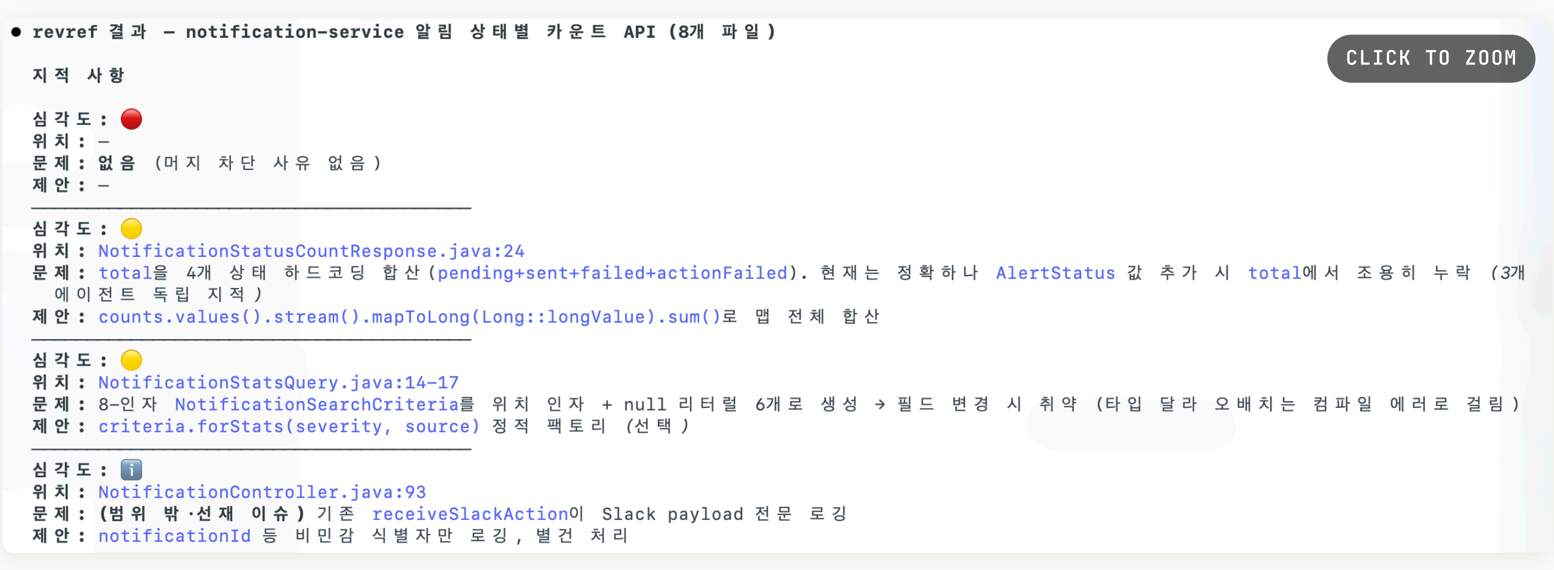
A. /revref — 컨벤션 준수 자동 검토

`git diff` 변경분(`--context`)이나 지정 파일을 대상으로, **차원별 리뷰 에이전트를 병렬 실행**해 위반을 심각도와 근거(`conventions §N`)와 함께 보고합니다. 같은 일반론 지적이 반복되면 규칙 문서로 흡수합니다.

병렬 검토 차원 — ① 아키텍처/레이어/DDD(§2,5,7) ② Spring/JPA 내부(§1) ③ 동시성(§6) ④ 타입 안정성·공통 모듈(§4,9). `--context` 모드에서는 정확성(`/code-review`)·보안(`/security-review`) 검토까지 통합해 **🔴/🟡 + 파일:라인 + 근거 → 제안** 형식으로 종합합니다.

/revref 실행 결과 — 차원별 병렬 검토 종합

WORKFLOW A



```
● revref 결과 — notification-service 알림 상태별 카운트 API (8개 파일)

  지적 사항

  심각도: 🔴
  위치: —
  문제: 없음 (머지 차단 사유 없음)
  제안: —
  ────────────────────────────────────
  심각도: 🟡
  위치: NotificationStatusCountResponse.java:24
  문제: total을 4개 상태 하드코딩 합산(pending+sent+failed+actionFailed). 현재는 정확하나 AlertStatus 값 추가 시 total에서 조용히 누락 (3개
    에이전트 독립 지적)
  제안: counts.values().stream().mapToLong(Long::longValue).sum()로 맵 전체 합산
  ────────────────────────────────────
  심각도: 🟡
  위치: NotificationStatsQuery.java:14-17
  문제: 8-인자 NotificationSearchCriteria를 위치 인자 + null 리터럴 6개로 생성 → 필드 변경 시 취약 (타입 달라 오배치는 컴파일 에러로 걸림)
  제안: criteria.forStats(severity, source) 정적 팩토리 (선택)
  ────────────────────────────────────
  심각도: ℹ️
  위치: NotificationController.java:93
  문제: (범위 밖·선재 이슈) 기존 receiveSlackAction이 Slack payload 전문 로깅
  제안: notificationId 등 비민감 식별자만 로깅, 별건 처리
```

변경분을 차원별 에이전트가 병렬 검토해 🔴/🟡 **심각도 · 파일:라인 · 근거(conventions §N)**와 함께 보고한다.

B. /apitest — 빌드부터 게이트웨이 시나리오까지

1-2 · 빌드 · 단위

gradlew

`build -x test` → `test` . 실패 시 컴파일/테스트 원인을 요약하고 **여기서 중단**, 결과 저장 후 실패 보고.

3-4 · 인프라 · 시나리오

docker · *.http

게이트웨이 `:8080/actuator/health` 폴링 후 `scenario*.http` 를 **게이트웨이 기준**으로 실행, 단계별 상태코드·본문으로 통과/실패 판정.

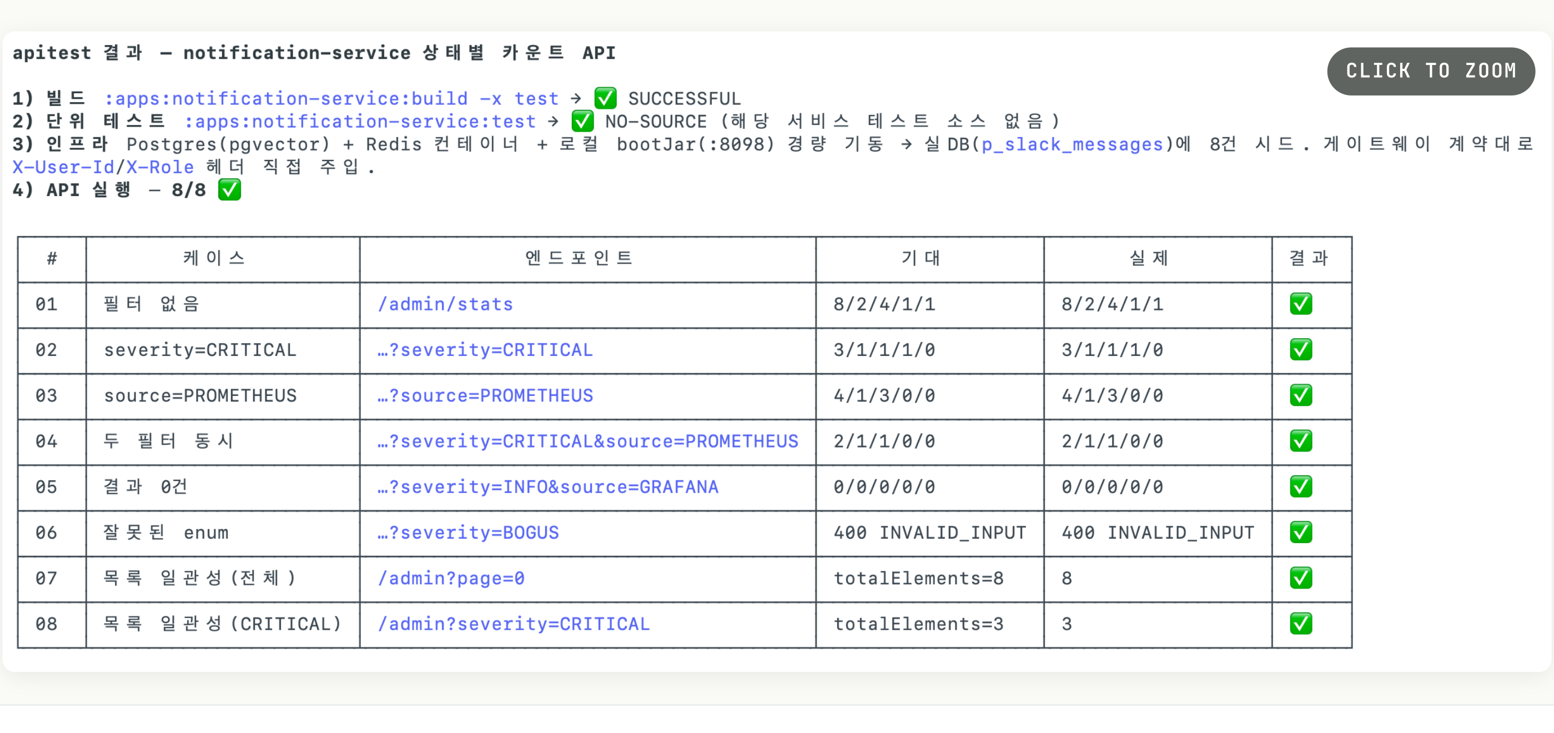
5 · 보존

tests/results

타임스탬프 폴더에 `summary.md` 표·요청·응답 저장. **토큰·시크릿은 마스킹**, 결과물은 `.gitignore` .

/apitest 실행 결과 — 빌드 → 테스트 → 게이트웨이 시나리오

WORKFLOW B



```
apitest 결과 — notification-service 상태별 카운트 API

1) 빌드 :apps:notification-service:build -x test → ✅ SUCCESSFUL
2) 단위 테스트 :apps:notification-service:test → ✅ NO-SOURCE (해당 서비스 테스트 소스 없음)
3) 인프라 Postgres(pgvector) + Redis 컨테이너 + 로컬 bootJar(:8098) 경량 기동 → 실 DB(p_slack_messages)에 8건 시드. 게이트웨이 계약대로
X-User-Id/X-Role 헤더 직접 주입.
4) API 실행 — 8/8 ✅
```

#	케이스	엔드포인트	기대	실제	결과
01	필터 없음	/admin/stats	8/2/4/1/1	8/2/4/1/1	✅
02	severity=CRITICAL	…?severity=CRITICAL	3/1/1/1/0	3/1/1/1/0	✅
03	source=PROMETHEUS	…?source=PROMETHEUS	4/1/3/0/0	4/1/3/0/0	✅
04	두 필터 동시	…?severity=CRITICAL&source=PROMETHEUS	2/1/1/0/0	2/1/1/0/0	✅
05	결과 0건	…?severity=INFO&source=GRAFANA	0/0/0/0/0	0/0/0/0/0	✅
06	잘못된 enum	…?severity=BOGUS	400 INVALID_INPUT	400 INVALID_INPUT	✅
07	목록 일관성(전체)	/admin?page=0	totalElements=8	8	✅
08	목록 일관성(CRITICAL)	/admin?severity=CRITICAL	totalElements=3	3	✅

빌드·단위 테스트 후 게이트웨이 헬스 폴링 → 시나리오(.http)를 단계별 상태코드·본문으로 **통과/실패 판정**한다.

C. notify-slack.sh — 작업을 깨지 않는 보고

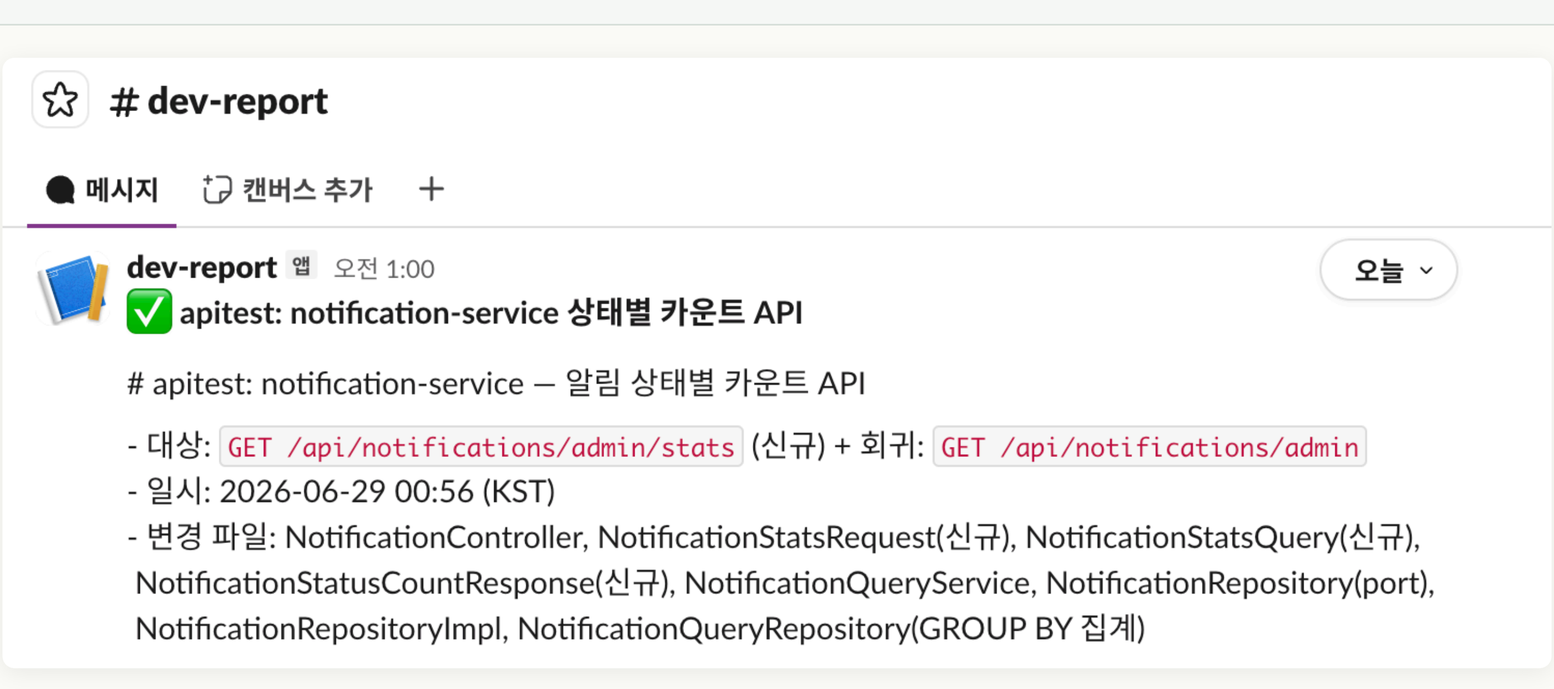
SCRIPTS/NOTIFY-SLACK.SH

```
# 성공/실패 + 제목 + 본문(또는 summary 파일)으로 Incoming Webhook 보고
scripts/notify-slack.sh --status success \
  --title "apitest: trade-service" \
  --file  "tests/results/20260628_153000/summary.md"

# 웹훅 미설정 시 → 경고만 출력하고 정상 종료 (빌드/작업을 깨지 않음)
```

외부 전송이므로 처음에는 보고 대상·내용을 사용자에게 확인받고, 사소한 변경(문서/오타)은 보고하지 않습니다.

notify-slack.sh 보고 — 작업 완료 Slack 알림

WORKFLOW C



dev-report 앱 오전 1:00

✅ apitest: notification-service 상태별 카운트 API

apitest: notification-service — 알림 상태별 카운트 API

- 대상: `GET /api/notifications/admin/stats` (신규) + 회귀: `GET /api/notifications/admin`
- 일시: 2026-06-29 00:56 (KST)
- 변경 파일: NotificationController, NotificationStatsRequest(신규), NotificationStatsQuery(신규), NotificationStatusCountResponse(신규), NotificationQueryService, NotificationRepository(port), NotificationRepositoryImpl, NotificationQueryRepository(GROUP BY 집계)

성공/실패 · 제목 · 요약을 Incoming Webhook으로 전송한다. 웹훅 미설정 혹은 슬랙 전송 오류 시 경고만 출력하고 **작업을 깨지 않는다.**

05 · GUARDRAILS · CONTEXT

가드레일과 컨텍스트를 설계한다

생성을 빠르게 하되, **위험한 자동화는 막고(가드레일)** AI가 **틀린 맥락으로 추측하지 않게(컨텍스트)** 합니다.

가드레일 — 무엇을 못 하게 하나

- **합격 판정은 사람이** — 검토·테스트는 자동화하되 통과 여부는 직접 결정, 🔴 항목은 반드시 수정 후 **빌드로 재검증**
- **외부 전송은 확인 후** — Slack 보고는 처음에 사용자 승인
- **시크릿·PII 마스킹** — 테스트 결과·로그에 JWT/ `KIS_*` / `*_API_KEY` /비밀번호 평문 금지(§8)
- 테스트 산출물(`tests/results/`)은 **커밋 대상 아님** (.gitignore)
- 신뢰 경계 — 도메인 서비스는 JWT를 직접 검증하지 않고 게이트웨이가 주입한 `X-User-Id` 만 신뢰

컨텍스트 — 무엇을 언제 보게 하나

- **SSOT 상호참조** — 세 문서가 서로를 가리켜 중복·표류 방지
- **설정 외부화 인지** — "이 레포에서 `application.yaml` 찾지 말 것", 실제 설정은 Config 레포(`dev`)
- **작업 유형별 참고문서 라우팅** — RAG 작업 → `rag-architecture.md` , 알림 작업 → `aiops-architecture.md`
- 휘발성 큰 정보(스키마·설정)는 문서에 박제하지 않고 그때그때 실측

06 · PRINCIPLE

왜 이렇게까지 하는가

AI 생성물은 빠르지만 **이 코드베이스의 맥락을 모른 채 그럴듯하지만 틀린 코드**를 자신 있게 내놓습니다. 그 실수가 코드베이스에 들어오지 못하도록 규칙(SSOT)·자동 검토/검증·가드레일로 **여러 겹의 관문**을 두고, 사람은 생성이 아니라 **감독과 최종 판단**에 시간을 씁니다. AI는 빨라지되 **사람의 통제 밖에서 코드를 확정하지 못합니다.** 이 관리 체계는 모델·도구가 바뀌어도 남습니다.

AI에게 개발을 일임하지 않는다 — 통제하고 감독한다.